
Model Selection & Evaluation

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Slides that adapt figures or content from other sources have a link to the source.

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Outline

- Recap of Supervised Learning and Importance of Generalization
- Model selection & evaluation techniques
 - Train-validation-test set
 - K-fold CV
- Evaluation Metrics
- Putting it all together
- Summary

Supervised Learning Setup [1]

- We have a real-world task. Can we apply supervised learning?
 1. Determine what are the input features of the task x_i .
 2. Determine what are the outputs of the task y_i . Is it a regression or classification task?
 3. Determine what is the evaluation metric that makes sense for the task. Is it accuracy, mean-squared-error, some other business related metric, etc. The metric should reflect a performance measure that you care about.
 4. Does a labelled dataset already exist, or do we have to gather it? The options if data does not exist are many and will not be discussed. We'll assume that a labelled dataset already exists.

Supervised Learning Setup [2]

- A **trained** machine learning algorithm is called a model, hypothesis, or classifier.
- Given a labelled dataset $S = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, the goal in supervised learning is to use the data to train a machine learning model that is able to **generalize** to **unseen** instances.
- **Generalization:** The ability of a model (hypothesis) to **correctly** predict the label of **unseen** instances.
- An **unseen** instance is an instance that the model did not train on. It is not part of the training data. This is the new instance x_{new} that we denote in class.

Supervised Learning Setup [3]

- **Generalization:** The ability of a model (hypothesis) to **correctly** predict the label of **unseen** instances.
- An important assumption in supervised learning is that: the data that is used for training comes from the same distribution as the unseen data.
- What can happen when this assumption is violated?
- If I train a model on some training data, and then I predict on new instances that come from a completely different distribution, then it is no surprise if the model does poorly.
- For example, my training data consists of high school students, but my unseen data consists of PhD students.

Supervised Learning Setup [4]

- We can estimate generalization of a specific model using a dataset and an evaluation metric like accuracy or mean squared error.
- A model's evaluation on the **training set** is biased since the training set was used to train the model. So of course it is expected to do well on the training set.
- Let's say we have three **trained** models A, B, and C. We can use a **validation set** to evaluate the three models and select the best one. This is called **model selection**.
- A test set that was never used in training or model selection in **any way** is the best estimate of a model's generalization performance.
- After selecting a final model via model selection, we can evaluate the performance of the final model on the test set. This is called **model evaluation**.

Main Takeaway

- The goal of supervised learning is to produce a model that is able to correctly predict the labels of unseen instances (generalization).
- You should use an appropriate evaluation metric for your task.
- A key assumption in supervised learning is that the data that was used for training the model comes from the same distribution as the data the model will see in production.
- Model selection is the process of selecting a final model for a learning task. There are many ways to do this, we will discuss two popular methods.
- You want to measure the generalization performance of your models. The best estimate for this is using a test set that was never used in any decision making.

Model Selection & Evaluation

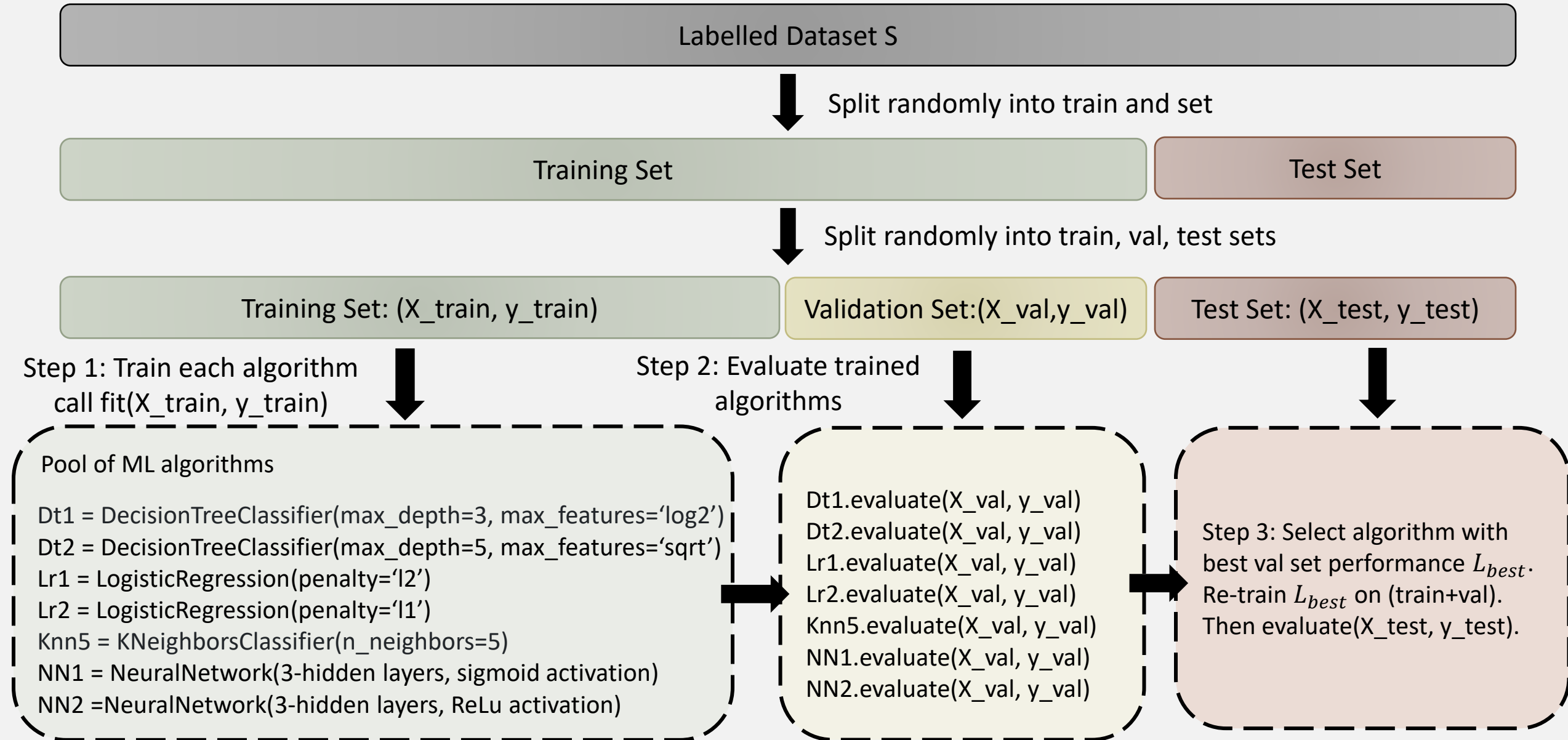
Model Selection Options

- I have a labelled dataset S , and I want to apply supervised learning. There are many ML algorithms, how do I choose one?
- To select a final model for a supervised learning task, there are two popular options to do this:
 1. Split your data into: Train-Validation-Test sets.
 2. Split your data into train and test sets. Use k-fold cross-validation (CV) on the training set to evaluate your algorithms.
- Note there are other ways to do model selection, but these two are popular methods.

Train-Validation-Test [1]

- Common split ratios for train-val-test is 60-20-20. Use uniform random splitting for regression problems, and stratified for classification problems.
1. Define a pool of ML algorithms M_{pool} . This pool includes various parameter setting for algorithms.
 2. Define a performance metric of interest.
 3. Train the algorithms in M_{pool} on the train set.
 4. Evaluate validation set performance of the **trained** models in M_{pool} . Select the algorithm L_{best} that gave you the best validation set performance.
 5. Re-train L_{best} on (train+val) data. Then evaluate on test set to get an estimate of the true performance.

Train-Validation-Test: Illustrated



Train-Validation-Test [2]

- Think of these three sets (train, val, test) as protective layers against overfitting.
- Recall that each ML algorithm is selecting from among its set of hypotheses the best hypothesis that fits the training data (or loss on the training data).
- For example, linear regression selects from among linear weighted hypothesis. Decision tree selects from among possible trees with feature splits.
- So, training on the train-set is used to select from among the many hypotheses in a single algorithm. The output hypothesis is the minimizer of an error on the train-set.
- Validation set is the first layer of protection against overfitting. It selects from among trained algorithms (output hypotheses).
- The test set is the last layer of protection against overfitting. You should not touch this until you have selected a final model. Evaluate your final model on the test set.

Train-Validation-Test [3]

- In step 1, we fit each algorithm in M_{pool} on the training set. Thus, each algorithm selects from its hypothesis set and produces a hypothesis h_i^* . The number of hypotheses considered in step 1 can be infinite.
- In step 2, we use the validation set to **select** among the **trained** hypotheses h_i^* . This process of selection is in some way **training** on the validation set. However, notice that the number of hypotheses we select from is reduced in step 2 $\{h_1^*, h_2^*, \dots, h_{|M_{pool}|}^*\}$
- The likelihood of overfitting on the validation set depends on $|M_{pool}|$ and the size of the validation set.
- This makes sense. The more algorithms we have in M_{pool} , the more chance there is that we get one that will do well on the specific validation set.

Theorem 11.2 Let $\mathcal{H} = \{h_1, h_2, \dots, h_{|M_{pool}|}\}$ be an arbitrary set of predictors and assume that the loss function is in $[0, 1]$. Assume that a validation set V of size m_V is sampled independent of H . Then:

$$\text{Prob} \left(\forall h \in \mathcal{H}, \underbrace{|trueerror(h) - valerror(h)|}_{\text{Difference between true error and validation set error. We want this to be small.}} \leq \underbrace{\sqrt{\frac{\ln(2|\mathcal{H}|/\delta)}{2m_V}}}_{\text{Error bound value. We can set this.}} \right) \geq \underbrace{1 - \delta}_{\text{Confidence. We can set this}}$$

- The difference between the true error and validation error is referred to as ϵ . We can set this, and as a result we get the following:

$$\epsilon = \sqrt{\frac{\ln(2|\mathcal{H}|/\delta)}{2m_V}} \longrightarrow m_V = \frac{\ln(2|\mathcal{H}|/\delta)}{2\epsilon^2}$$

Adapted from: [book pg. 147](#)

$$\text{Prob}(\forall h \in \mathcal{H}, \quad |trueerror(h) - valerror(h)| \leq \epsilon) \geq 1 - \delta \qquad m_V = \frac{\ln(2|\mathcal{H}|/\delta)}{2\epsilon^2}$$

- So for example, if we have 10 trained models, and we set $\delta = 0.05$ and $\epsilon = 0.01$. Then this is saying that if we want to be $(1 - \delta) = 95\%$ confident that the model we select will have a true error and val-error difference no more than $\epsilon = 0.01$, then we need a validation set size of at least:

$$m_V \geq \frac{\ln(2|\mathcal{H}|/\delta)}{2\epsilon^2} = \frac{\ln(2(10)/0.05)}{2(0.01^2)} = 29958$$

- Most of the cost is from $\frac{1}{\epsilon^2}$; tighter error bound requires more examples. The good thing is that the cost from number of models is logarithmic in $|\mathcal{H}|$.

Adapted from: [book pg. 147](#)

This theorem tells us that the error on the validation set approximates the true error as long as $\mathcal{H}$ is not too large. However, if we try too many methods (resulting in $|\mathcal{H}|$ that is large relative to the size of the validation set) then we're in danger of overfitting. From: [book pg. 148](#)

- Furthermore, in case we do overfit, we have the final protection layer. We use the test set to evaluate the performance of the final selected model.

Theorem Let h be some predictor and assume that the loss function is in $[0, 1]$. Assume that a test set Z of size m_Z is sampled independent of h . Then:

$$\text{Prob} \left(|trueerror(h) - testerror(h)| \leq \sqrt{\frac{\ln(2/\delta)}{2m_Z}} \right) \geq 1 - \delta$$

Notice we get a tighter error bound since we only have one hypothesis

Adapted from: [book pg. 148](#)

K-Fold CV [1]

- Using one train-val split, there is a chance that our random splitting resulted in an easy validation set.
- In some cases, we might have a small dataset to begin with. What options are there in this case?
- Split the dataset S into train-test sets. Then use K-fold Cross-validation on the training set.
- This method splits the training set into K sets of the same size: $S_1, S_2, S_3, \dots, S_K$.
- So, if the training set has m instances, then each S_i has about $\frac{m}{K}$ instances.

K-Fold CV [2]

- K-fold CV is also used for model selection. As follows:
 1. Define a pool of ML algorithms M_{pool} . This pool includes various parameter setting for algorithms.
 2. Define a performance metric of interest.
 3. Split data into training and test sets.
 4. Use K-fold CV on the training set to compute the CV metrics on the algorithms in M_{pool} . Select the algorithm that gave the best CV performance L_{best} .
 5. Re-train L_{best} on training set. Then evaluate on test set to get an estimate of the true performance.

K-Fold CV: Illustrated [1]

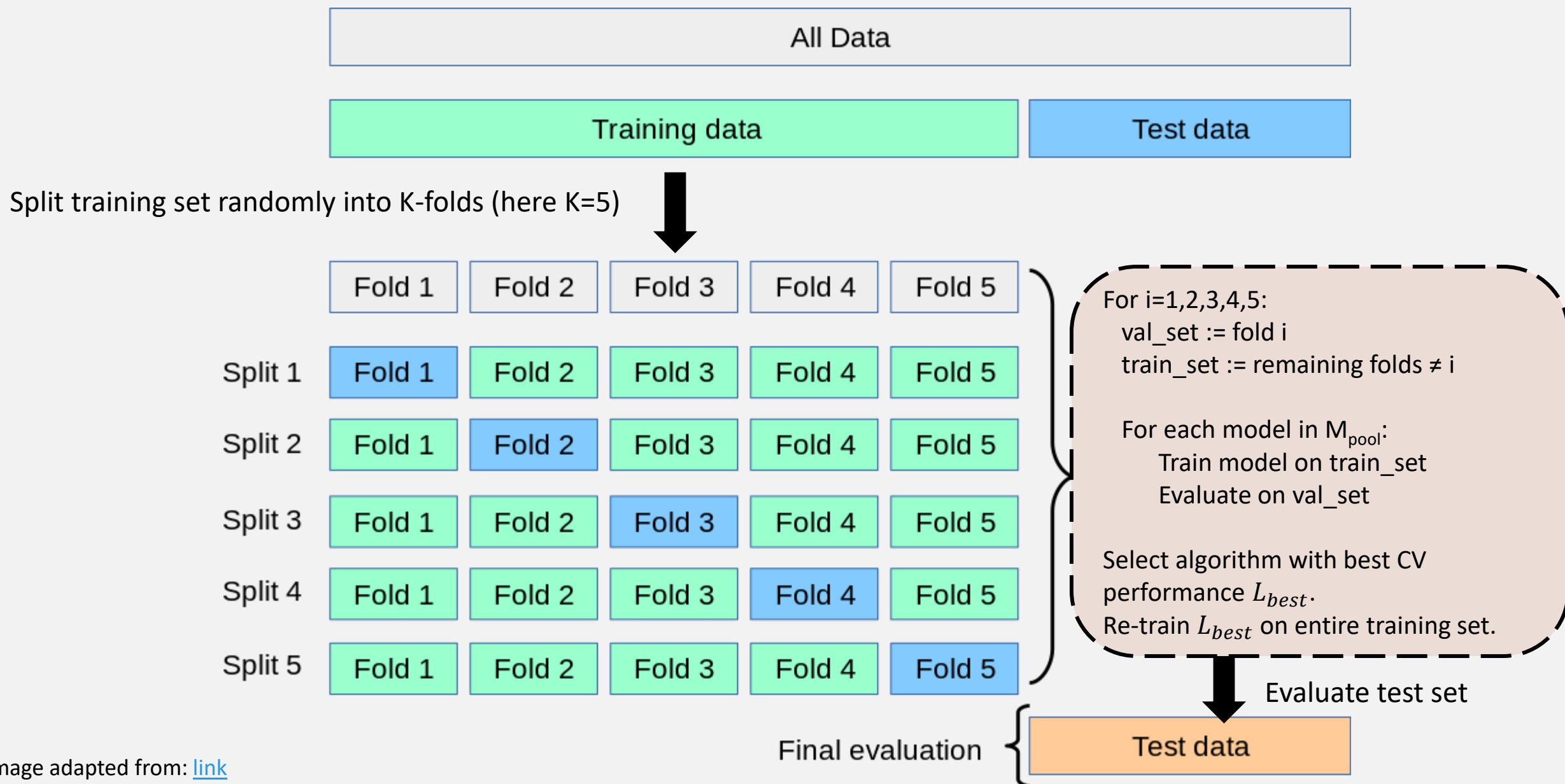


Image adapted from: [link](#)

Kfold CV Model Selection Example

Pool of ML algorithms

Dt = DecisionTreeClassifier(max_depth=3, max_features='log2')

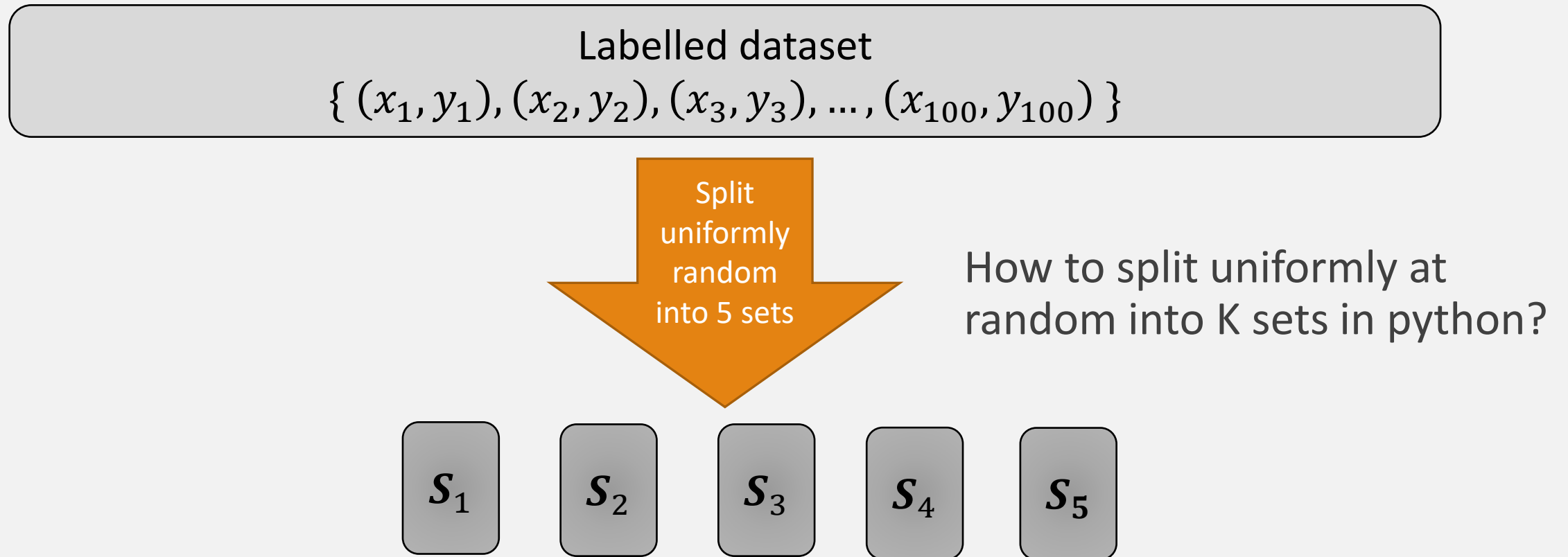
Lr = LogisticRegression(penalty='l2')

NN = NeuralNetwork(3-hidden layers, sigmoid activation)

Iteration	Train Set	Validation Set	Val performance on Trained DT	Val performance on Trained LR	Val performance on Trained NN
1	S_2, S_3, S_4, S_5	S_1	11/20 correct	10/20 correct	19/20 correct
2	S_1, S_3, S_4, S_5	S_2	17/20 correct	9/20 correct	18/20 correct
3	S_1, S_2, S_4, S_5	S_3	16/20 correct	11/20 correct	17/20 correct
4	S_1, S_2, S_3, S_5	S_4	13/20 correct	8/20 correct	18/20 correct
5	S_1, S_2, S_3, S_4	S_5	16/20 correct	19/20 correct	17/20 correct
			73%.	57%.	89%.

Select the algorithm with the highest average performance. So, in this case the NN algorithm. Retrain the NN algorithm on entire training set ($S_1+S_2+S_3+S_4+S_5$), then evaluate on the held aside test set.

Splitting the dataset Kfold CV: Regression Task



- How many instances does each set have? $100/5 = 20$

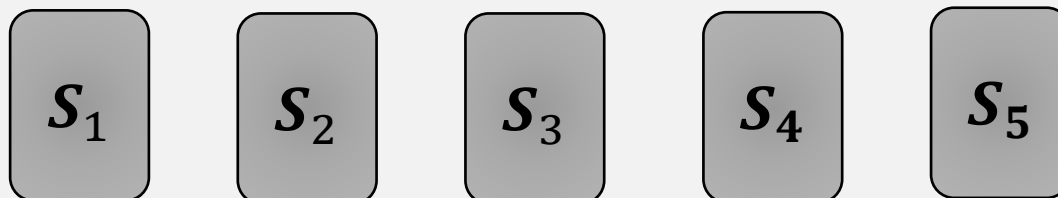
Splitting the dataset Kfold CV: Classification Task

Labelled dataset with 30 $+$ labels and 70 $-$ labels.

$\{ (x_1, +), (x_2, -), (x_3, -), \dots, (x_{100}, +) \}$

Split
stratified
random
into 5 sets

How to split stratified
random into K sets in
python?



- How many instances does each set have? $100/5 = 20$
- How many $+$ instances does each set have? $30/5 = 6$.
- How many $-$ instance does each set have? $70/5 = 14$

K-Fold CV: Remarks [1]

- Using one validation set, there is a chance that our random splitting resulted in an *easy* validation set.
- To remedy the reliance on one train-val split, Kfold CV evaluates an algorithm on various train-val splits and computes the average performance.
- You may also want to consider the variance of the CV performance, not just the average. Why?
- In theory, larger values for K give stronger estimation guarantees. But in practice, this would mean iterating many times, retraining models, which can be time-consuming for some models (deep learning).
- What value should we use for K? Rule of thumb is 5 to 10.

K-Fold CV: Remarks [2]

- What should I use in practice? A single train-val-test split or Kfold CV?
- If your labelled dataset is large and covers most of the underlying data distribution. For example, the resulting train-val-test split gives a large validation set, then this should be safe to use. Note you can use the bounds we talked about earlier.
- Sometimes training takes a long time for a complex algorithm, either due to the size of the training set or complexity of the computations. In this case, Kfold CV may not be practically feasible.
- Sometimes you have a small dataset to begin with (like <1000 examples), so Kfold CV is better option.
- As a recommendation, use Kfold CV with $K=10$ whenever possible.

Evaluation Metrics

Metrics in scikit-learn for regression tasks

- Scikit-learn implements common regression metrics. See the [documentation](#).
- Each metric has pros and cons in terms of what it evaluates. For example, the squared error penalizes outliers more than absolute error.
- You can also implement your own custom metric that you care about.

Regression	
'explained_variance'	<code>metrics.explained_variance_score</code>
'max_error'	<code>metrics.max_error</code>
'neg_mean_absolute_error'	<code>metrics.mean_absolute_error</code>
'neg_mean_squared_error'	<code>metrics.mean_squared_error</code>
'neg_root_mean_squared_error'	<code>metrics.mean_squared_error</code>
'neg_mean_squared_log_error'	<code>metrics.mean_squared_log_error</code>
'neg_median_absolute_error'	<code>metrics.median_absolute_error</code>
'r2'	<code>metrics.r2_score</code>
'neg_mean_poisson_deviance'	<code>metrics.mean_poisson_deviance</code>
'neg_mean_gamma_deviance'	<code>metrics.mean_gamma_deviance</code>
'neg_mean_absolute_percentage_error'	<code>metrics.mean_absolute_percentage_error</code>

Metrics in classification tasks

- Classification metrics are interesting, and you should carefully consider which classification metric to use based on the properties of your problem.
- One main property to consider is the class skew. That is, how often do the labels appear relative to each other.
- In some problems, the label 1 can be extremely rare compared to the label 0.
- Using accuracy for such problems is not helpful for determining good models.
- The good news is that there are other more suitable classification metrics in such cases.
- We'll discuss binary classification tasks, but you can generalize our discussion to multiclass tasks.

Metrics in scikit-learn for classification tasks

- Scikit-learn implements common classification metrics. See the [documentation](#).
- The ones you should definitely know about are accuracy, ROC-AUC, and Average Precision-Recall (also called average precision).

Classification	
'accuracy'	<code>metrics.accuracy_score</code>
'balanced_accuracy'	<code>metrics.balanced_accuracy_score</code>
'top_k_accuracy'	<code>metrics.top_k_accuracy_score</code>
'average_precision'	<code>metrics.average_precision_score</code>
'neg_brier_score'	<code>metrics.brier_score_loss</code>
'f1'	<code>metrics.f1_score</code>
'f1_micro'	<code>metrics.f1_score</code>
'f1_macro'	<code>metrics.f1_score</code>
'f1_weighted'	<code>metrics.f1_score</code>
'f1_samples'	<code>metrics.f1_score</code>
'neg_log_loss'	<code>metrics.log_loss</code>
'precision' etc.	<code>metrics.precision_score</code>
'recall' etc.	<code>metrics.recall_score</code>
'jaccard' etc.	<code>metrics.jaccard_score</code>
'roc_auc'	<code>metrics.roc_auc_score</code>
'roc_auc_ovr'	<code>metrics.roc_auc_score</code>
'roc_auc_ovo'	<code>metrics.roc_auc_score</code>
'roc_auc_ovr_weighted'	<code>metrics.roc_auc_score</code>
'roc_auc_ovo_weighted'	<code>metrics.roc_auc_score</code>

Confusion Matrix (also known as Contingency Table) [1]

		Actual class	
		Positive	Negative
Predicted class	Positive	True Positive (TP)	False Positive (FP) Type I Error
	Negative	False Negative (FN) Type II Error	True Negative (TN)

Total # positive instances = TP + FN Total # negative instances = TN + FP

- For a binary classification problem, we can refer to the labels as 0 and 1, or -1 and 1, or negative and positive.
- A **confusion matrix** showcases the errors made by a model on a labelled dataset.
- These 4 numbers allows us to compute various classification metrics like accuracy, ROC, and Precision-Recall.
- True positive: the number of instances the model *predicted* as positive, and the true label is positive. This is a **correct** prediction.
- True negative: the number of instances the model *predicted* as negative, and the true label is negative. This is a **correct** prediction.
- False positive: the number of instances the model *predicted* as positive, but the true label is negative. This is also called a **type I error**.
- False negative: the number of instances the model predicted as negative, but the true label is positive. This is also called a **type II error**.

Confusion Matrix [2]

Test set					
feature 1	feature2	True label	model A prediction	model B prediction	model C prediction
0.11	0.21	1	1	1	1
0.70	0.05	0	0	1	0
0.27	0.23	0	0	1	0
0.63	0.78	1	0	1	1
0.90	0.36	0	1	0	0
0.90	0.95	0	1	0	0
0.25	0.22	1	0	0	1

Model **A** Confusion Matrix
on test set

Actual class

Predicted
class

	Actual class	
	Positive	Negative
Positive	TP: 1	FP: 2
Negative	FN: 2	TN: 2

Model **B** Confusion Matrix
on test set

Actual class

Predicted
class

	Actual class	
	Positive	Negative
Positive	TP: 2	FP: 2
Negative	FN: 1	TN: 2

Model **C** Confusion Matrix
on test set

Actual class

Predicted
class

	Actual class	
	Positive	Negative
Positive	TP: 3	FP: 0
Negative	FN: 0	TN: 4

Confusion Matrix [2]: accuracy

		Actual class	
		Positive	Negative
Predicted class	Positive	True Positive (TP)	False Positive (FP) Type I Error
	Negative	False Negative (FN) Type II Error	True Negative (TN)

Total # positive instances = TP + FN Total # negative instances = TN + FP

$$\text{accuracy} = \frac{TP + TN}{TP + FP + FN + TN} = \frac{\# \text{ correct predictions in set}}{\# \text{ instances in set}}$$

$$\text{error rate} = 1 - \text{accuracy} = 1 - \frac{TP + TN}{TP + FP + FN + TN}$$

Accuracy when there is a class skew

- Consider this binary classification two-feature dataset.
- From this data, we see that the label 1 appears very rarely.
- What is the accuracy of a simple model that outputs 0 for all input?
- It would be 95% accuracy.
- However, such a simple model did not learn the relationship between input and output.
- So, we need a better metric than accuracy when we have such large class skew (here label 0 appears much more than label 1).

feature 1	feature 2	output
0.21	0.26	1
0.30	0.40	0
0.14	0.17	0
0.43	0.38	0
0.36	0.91	0
0.01	0.66	0
0.93	0.90	0
0.81	0.51	0
0.37	0.23	0
0.77	0.99	0
0.42	0.61	0
0.07	0.56	0
0.98	0.80	0
0.14	0.29	0
0.22	0.46	0
0.26	0.38	0
0.60	0.59	0
0.96	0.75	0
0.21	0.26	0
0.00	0.20	0

When is accuracy not appropriate?

- We saw from the example that accuracy is not suitable when there is a large class skew. That is, one class appears much more frequently than the others.
- Another situation is when you want to consider misclassification costs. That is, the cost of a false positive and the cost of a false negative are different.
- For example, in a medical diagnosis, the cost of a false positive is just an extra lab test. But the cost of a false negative can be fatal. So, you would like to use a metric that penalizes or highlights false negatives more.
- So, if accuracy is not appropriate for these use-cases, what other metrics can we use?

Confusion Matrix [2]: TPR and FPR

		Actual class	
		Positive	Negative
Predicted class	Positive	True Positive (TP)	False Positive (FP) Type I Error
	Negative	False Negative (FN) Type II Error	True Negative (TN)

Total # positive instances = TP + FN Total # negative instances = TN + FP

$$\text{True Positive Rate (TPR or Recall)} = \frac{TP}{\# \text{ actual positives}} = \frac{TP}{TP + FN}$$

$$\text{False Positive Rate (FPR)} = \frac{FP}{\# \text{ actual negatives}} = \frac{FP}{TN + FP}$$

Ideally, you want to maximize TPR and minimize FPR.

ROC Curves [1]

- Note that it's easy to maximize TPR alone. Just use a simple model that predicts positive class for all input. In such a case, $TPR = 1$ (good), however $FPR = 1$ (bad).
- Thus, to better evaluate a model you should measure both TPR and FPR.
- Some models like logistic regression can output the probability of the positive class (prediction is a value between 0 and 1). To force a prediction, we apply a threshold.
- A **Receiver-Operating-Characteristic (ROC)** curve plots the TPR vs. the FPR of a model as we vary the prediction threshold.
- The ROC measures the *overall* ability of a model to distinguish (separate) between the positive and negative classes.
- It does this via the TPR and FPR. A good model would maximize TPR (positive class identification) and minimize FPR (negative class error).

ROC Curves [2]

- Consider the dataset below where we predicted the probability using a trained model D. How can we plot the ROC curve?

test set			
Feature 1	Feature 2	True Label	Model D Predicted Prob($y = 1 \mid x$)
2.48	-2.48	1	0.67
1.18	-0.99	1	0.71
-2.08	1.26	0	0.04
0.36	0.75	0	0.62
-0.41	1.25	1	0.3
1.09	0.62	1	0.77
0.17	2.82	1	0.6
-2.14	-0.48	0	0.04
-0.32	-1.55	0	0.36
1.81	-1.4	1	0.71
-0.78	0.49	1	0.21
-2.31	1.12	1	0.03
-0.35	-0.34	1	0.34
-0.23	-1.77	0	0.4
-0.62	0.91	0	0.24
0.32	2.57	1	0.65
-1.7	1.19	0	0.07
-0.93	0.94	0	0.17
1.18	0.48	0	0.74
-1.66	1.06	0	0.07

Sort by probability score (largest to smallest)

test set			
Feature 1	Feature 2	True Label	Model D Predicted Prob($y = 1 \mid x$)
1.09	0.62	1	0.77
1.18	0.48	0	0.74
1.18	-0.99	1	0.71
1.81	-1.4	1	0.71
2.48	-2.48	1	0.67
0.32	2.57	1	0.65
0.36	0.75	0	0.62
0.17	2.82	1	0.6
-0.23	-1.77	0	0.4
-0.32	-1.55	0	0.36
-0.35	-0.34	1	0.34
-0.41	1.25	1	0.3
-0.62	0.91	0	0.24
-0.78	0.49	1	0.21
-0.93	0.94	0	0.17
-1.7	1.19	0	0.07
-1.66	1.06	0	0.07
-2.08	1.26	0	0.04
-2.14	-0.48	0	0.04
-2.31	1.12	1	0.03

ROC Curves [3]

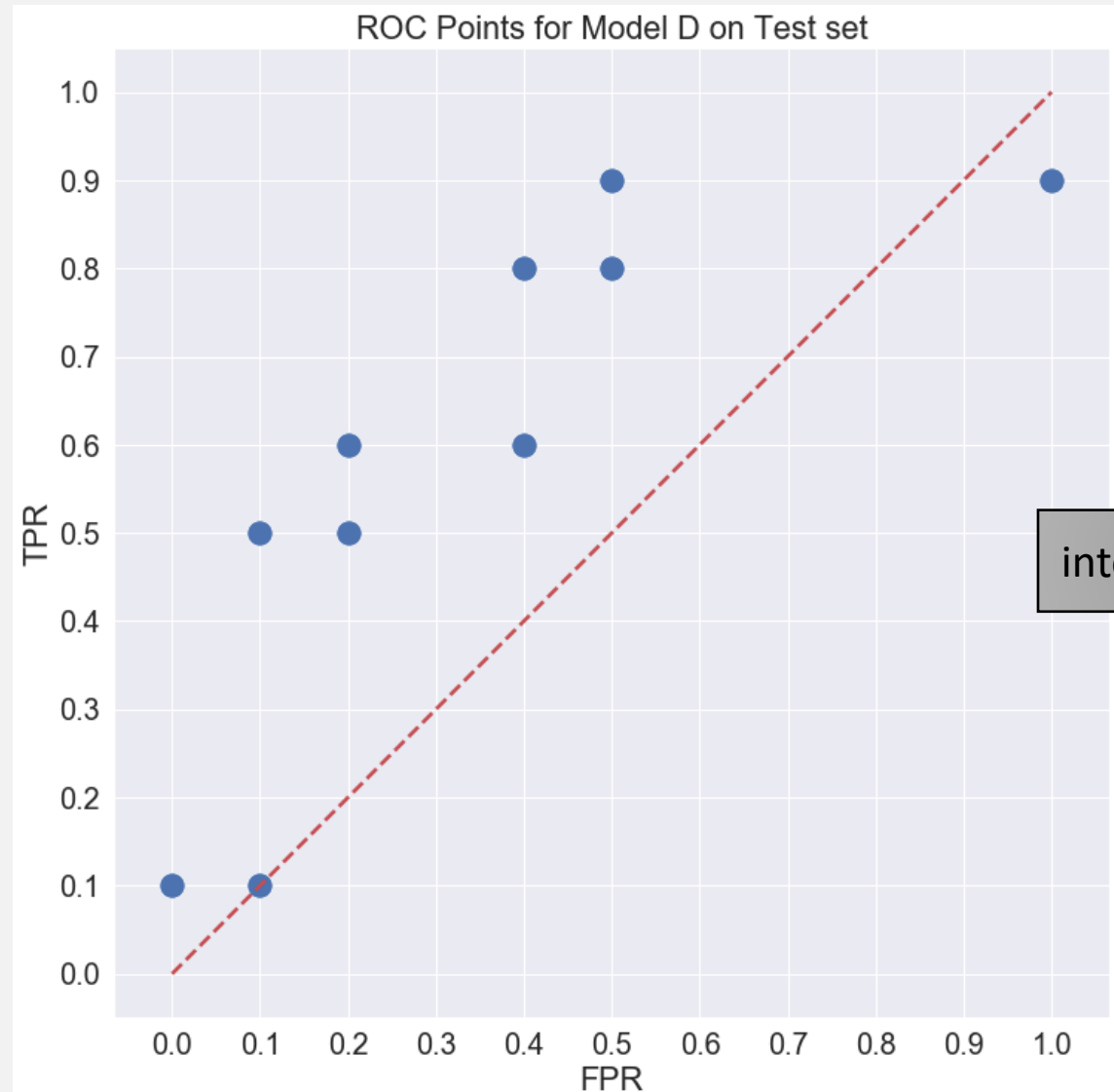
- After sorting, determine candidate thresholds by looking at where the true labels shift.

test set		True Label	Model D Predicted Prob($y = 1 \mid x$)	
Feature 1	Feature 2			
1.09	0.62	1	0.77	Threshold 0.77, TPR = 1/10, FPR = 0/10.
1.18	0.48	0	0.74	Threshold 0.74, TPR = 1/10, FPR = 1/10.
1.18	-0.99	1	0.71	
1.81	-1.4	1	0.71	
2.48	-2.48	1	0.67	
0.32	2.57	1	0.65	Threshold 0.65, TPR = 5/10, FPR = 1/10.
0.36	0.75	0	0.62	Threshold 0.62, TPR = 5/10, FPR = 2/10.
0.17	2.82	1	0.6	Threshold 0.6, TPR = 6/10, FPR = 2/10.
-0.23	-1.77	0	0.4	
-0.32	-1.55	0	0.36	Threshold 0.36, TPR = 6/10, FPR = 4/10.
-0.35	-0.34	1	0.34	
-0.41	1.25	1	0.3	Threshold 0.3, TPR = 8/10, FPR = 4/10.
-0.62	0.91	0	0.24	Threshold 0.24, TPR = 8/10, FPR = 5/10.
-0.78	0.49	1	0.21	Threshold 0.21, TPR = 9/10, FPR = 5/10.
-0.93	0.94	0	0.17	
-1.7	1.19	0	0.07	
-1.66	1.06	0	0.07	
-2.08	1.26	0	0.04	
-2.14	-0.48	0	0.04	Threshold 0.04, TPR = 9/10, FPR = 10/10.
-2.31	1.12	1	0.03	

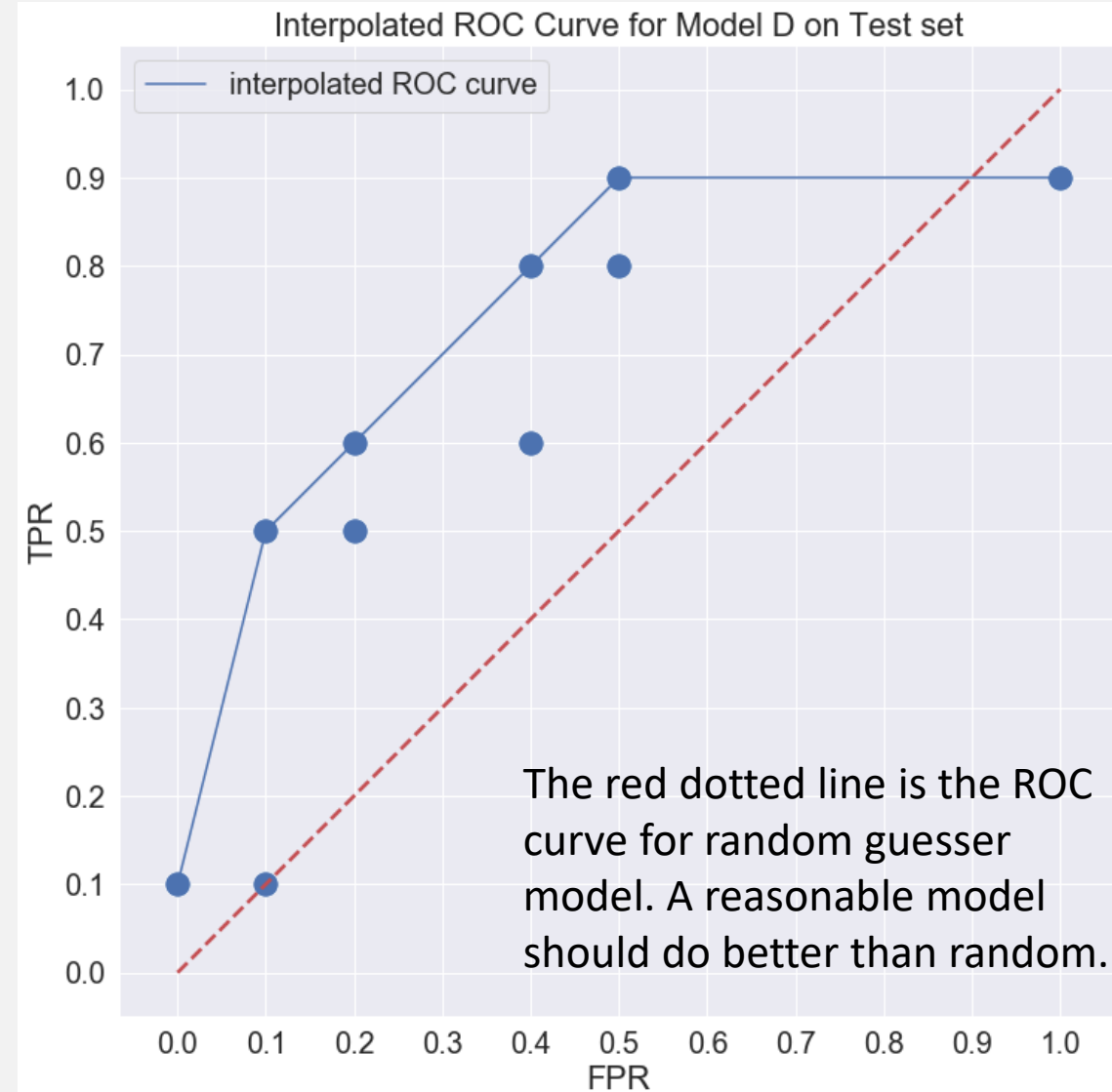
As we vary the threshold and compute the TPR and FPR, we can see how well the model separates the two classes. A *good* model will have a clear separation between the two classes.

ROC Curves [4]

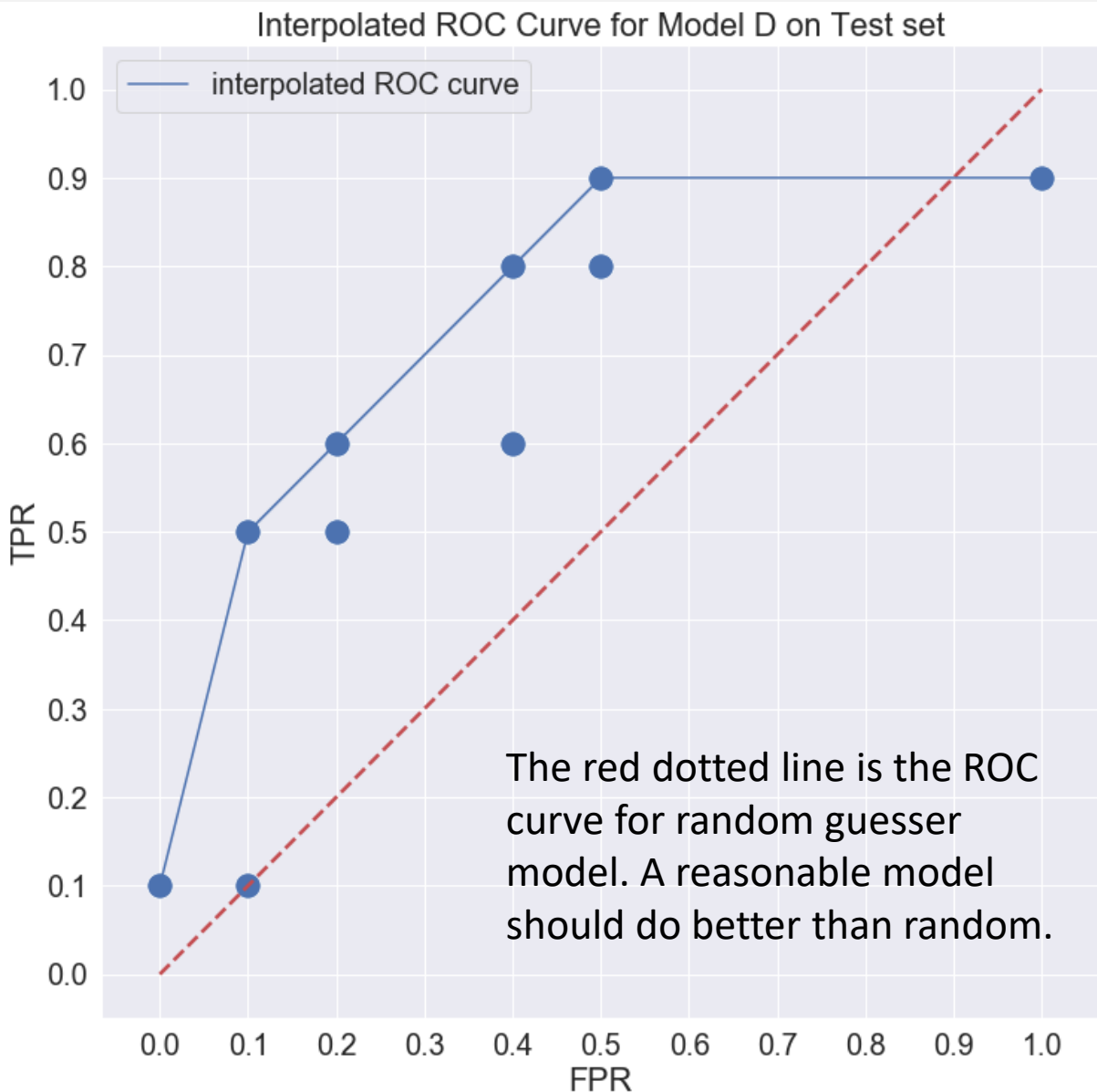
- We can plot these TPR and FPR points, and then interpolate to get an ROC curve.



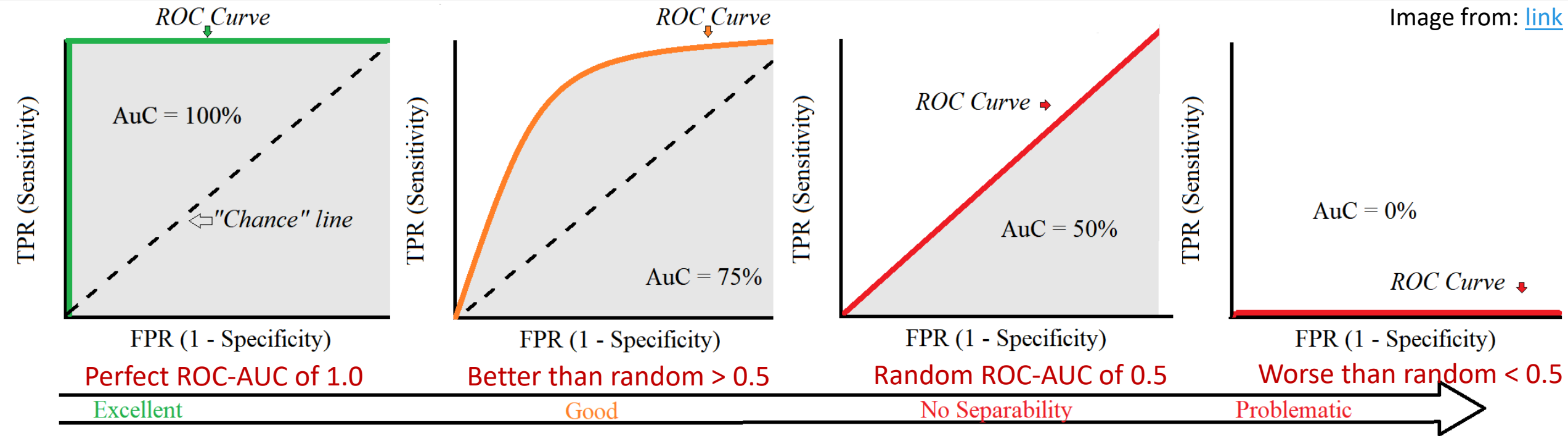
interpolate



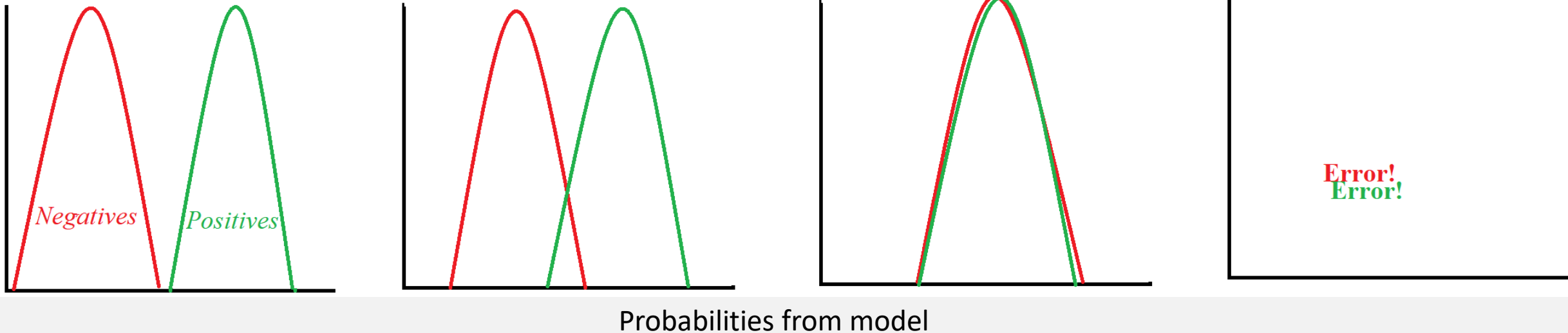
ROC Curves [5]

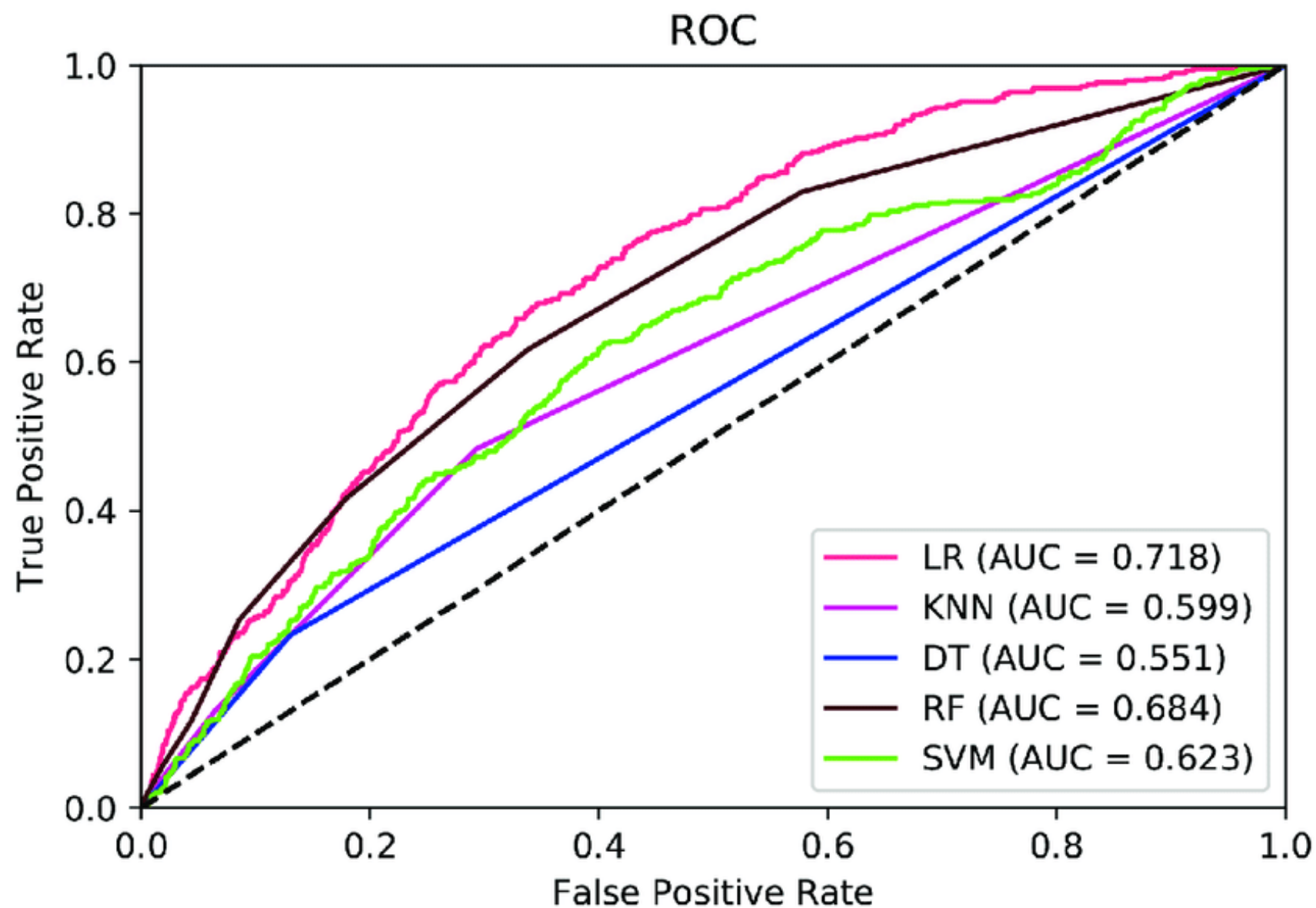


- Once we have the ROC curve, how do we get a single measure?
- We can compute the area under the ROC curve (ROC-AUC).
- Recall from calculus we can do this via summation on intervals under the curve.
- We can use scikit-learn's implemented ROC-AUC function: `roc_auc_score(y_test, y_pred_on_test)`
- This gives an ROC-AUC of 0.71
- The ROC-AUC of a random guesser (red dotted line) is 0.5
- So, a reasonable model should have ROC-AUC better than 0.5



Overlap = How well the model separates Negatives and Positives





- Using ROC-AUC, we can compare different models.
- From the ROC-AUC metrics, we see that the LR model (logistic regression) has the highest ROC-AUC.
- You can use sklearn's ROC plotting function [link](#)

Does ROC solve the class skew?

- We said that ROC-AUC measures how well a model separates the positive and negative classes.
- Does ROC-AUC account for a large class skew?
- Consider if we have a model with TPR = 0.9 and FPR = 0.01 on various datasets. This is desired since we have a high TPR and a low FPR. Now consider the following scenarios:

$$\text{TPR} = \frac{TP}{\# \text{ actual positives}} = \frac{TP}{TP + FN}$$

$$\text{FPR} = \frac{FP}{\# \text{ actual negatives}} = \frac{FP}{TN + FP}$$

$$\text{Precision} = \frac{TP}{\# \text{ predicted positives}} = \frac{TP}{TP + FP}$$

	fraction of positive class in the dataset (f_p)	Model's TPR	Model's FPR	Model's Precision: fraction of positive predictions that are correct
Dataset A	0.5	0.9	0.01	0.989
Dataset B	0.1	0.9	0.01	0.909
Dataset C	0.01	0.9	0.01	0.476
Dataset D	0.001	0.9	0.01	0.083

$$\text{TPR} = \frac{TP}{\#P} = 0.9$$

$$\text{FPR} = \frac{FP}{\#N} = 0.01$$

$$\frac{\#P}{\#P + \#N} = f_p$$

$$\text{Precision} = \frac{0.9\#P}{0.9\#P + 0.01\#N}$$

$$= \frac{0.9}{0.9 + 0.01 \frac{1-f_p}{f_p}}$$

- We see that if the fraction of positive class gets lower (rarer), TPR and FPR does not capture the performance of a model on the rare positive class (i.e. how precise it is on getting the rare class correct). Is there another metric we can use?

Confusion Matrix [3]: Precision and Recall

		Actual class	
		Positive	Negative
Predicted class	Positive	True Positive (TP)	False Positive (FP) Type I Error
	Negative	False Negative (FN) Type II Error	True Negative (TN)

Total # positive instances = TP + FN Total # negative instances = TN + FP

$$\text{True Positive Rate (TPR or Recall)} = \frac{TP}{\# \text{ actual positives}} = \frac{TP}{TP + FN}$$

$$\text{Precision} = \frac{TP}{\# \text{ predicted positives}} = \frac{TP}{TP + FP}$$

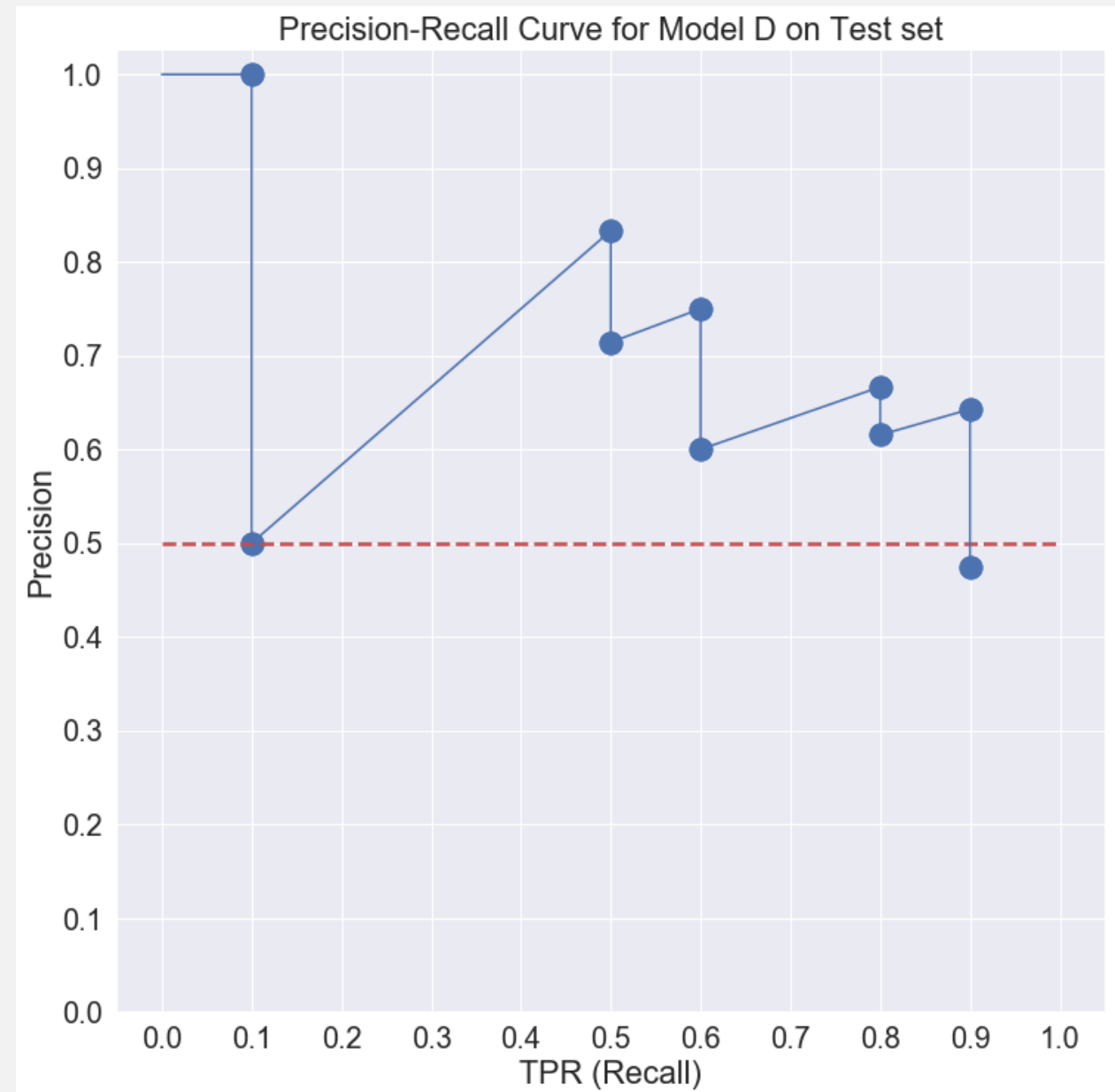
Ideally, you want to maximize Recall (TPR) and maximize Precision.

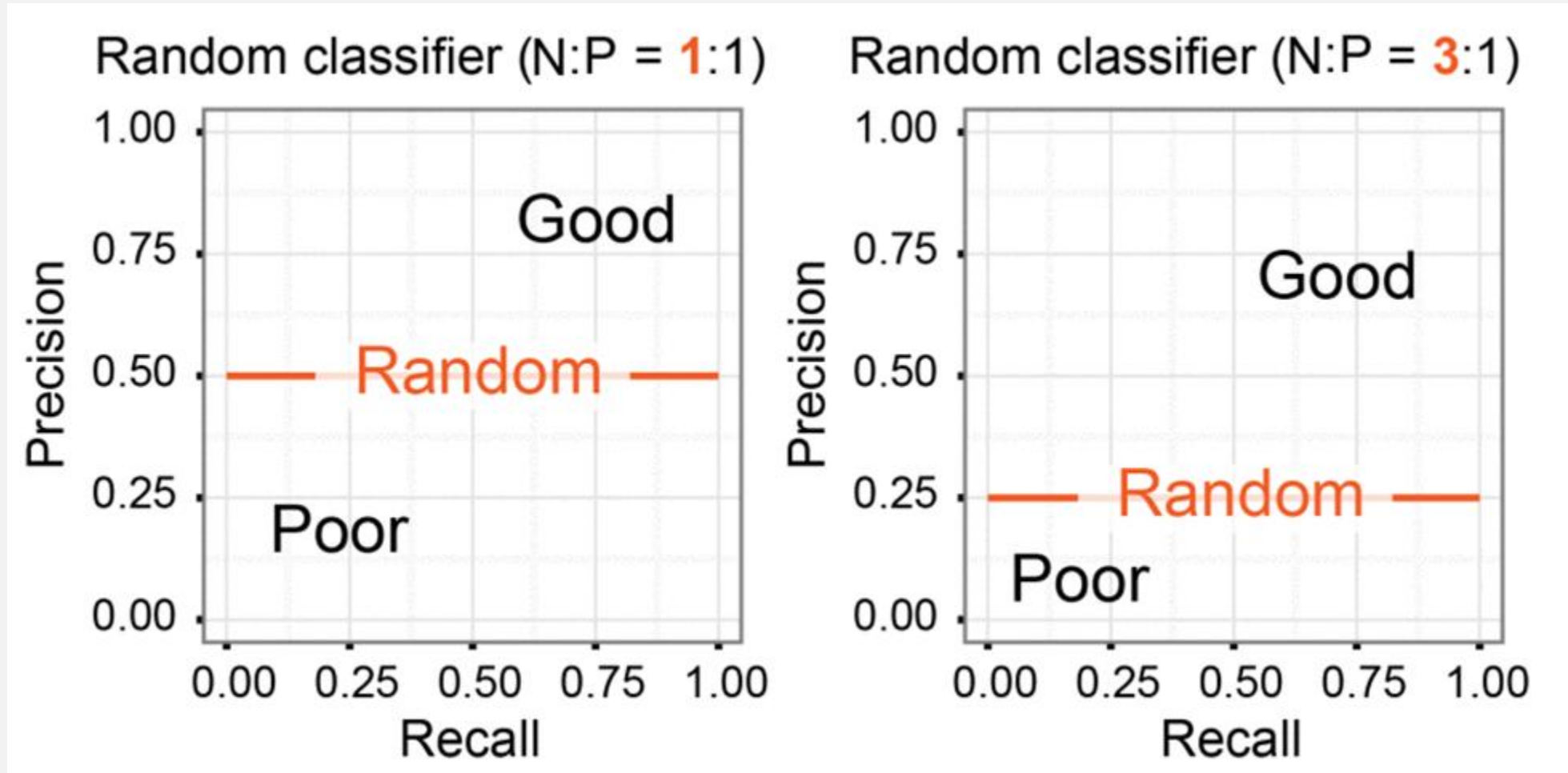
Precision-Recall Curves [1]

- Note that it's easy to maximize Recall (TPR) alone. Just use a simple model that predicts positive class for all input. In such a case, $\text{Recall (TPR)} = 1$ (good), however $\text{Precision} = \text{fraction of positive instances in dataset}$ (which can be low).
- If you have a large a class skew in your dataset (the positive class is rare), Precision-Recall is a good performance measure to determine how well your model is identifying the rare class.
- You can plot the Precision vs. Recall curve of a model as we vary the prediction threshold.
- Then we can get the AUC of Precision-Recall curve (this is also called average precision).

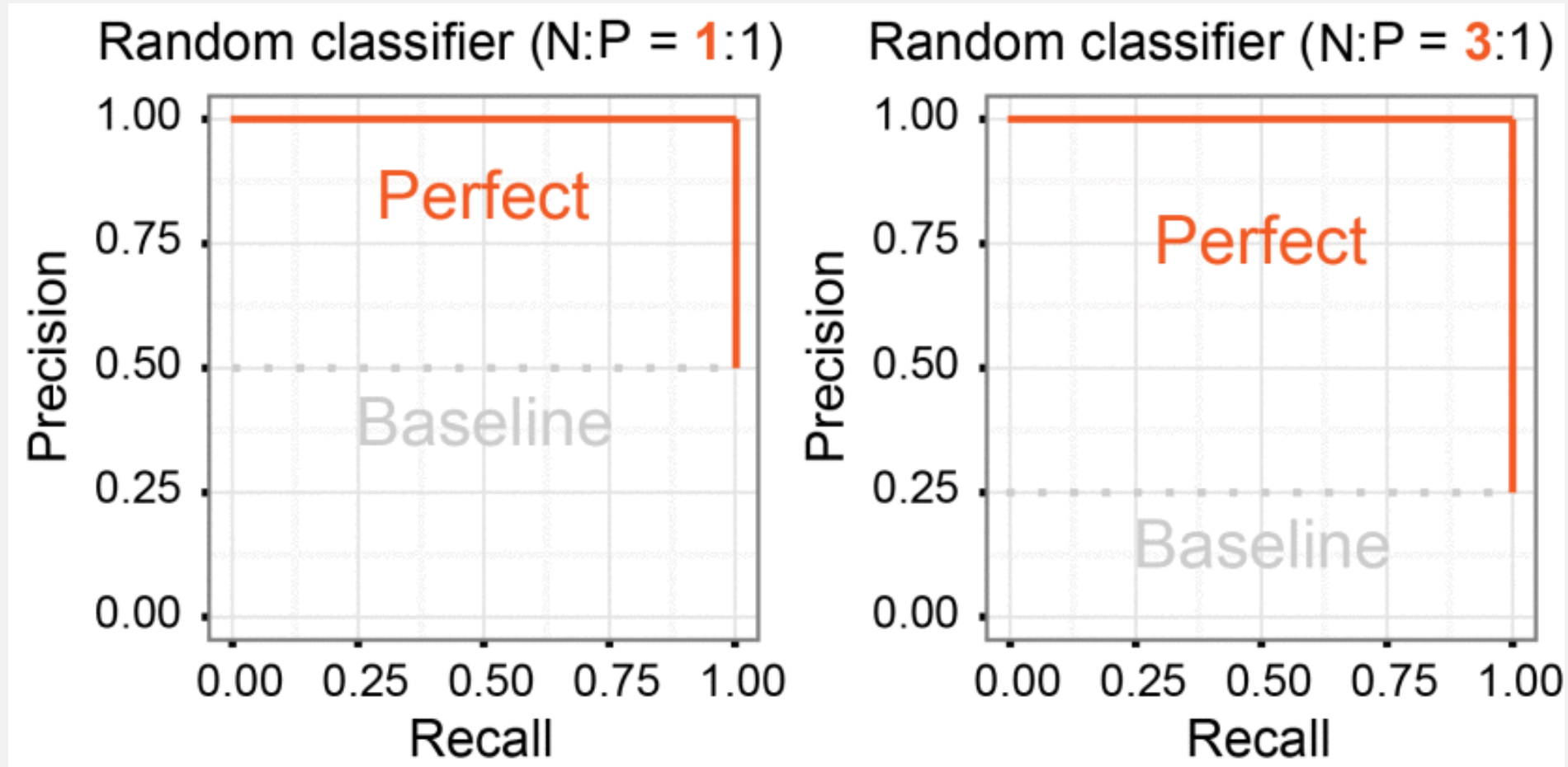
Precision-Recall Curves [2]

- You apply the same process as ROC-AUC: sort the instances by the probability, determine candidate thresholds at shifting labels, compute Precision-Recall at each threshold, plot the Precision-Recall points, interpolate, then compute AUC.
- The red dotted line represents a baseline model. The baseline precision is the fraction of positive instances in the dataset. For the dataset we introduced in [slide](#), we have 10 pos. and 10 neg.
- You want models to get a precision-recall AUC better than the baseline precision-recall AUC.
- For a dataset with extremely rare positive class, the baseline precision will be low.

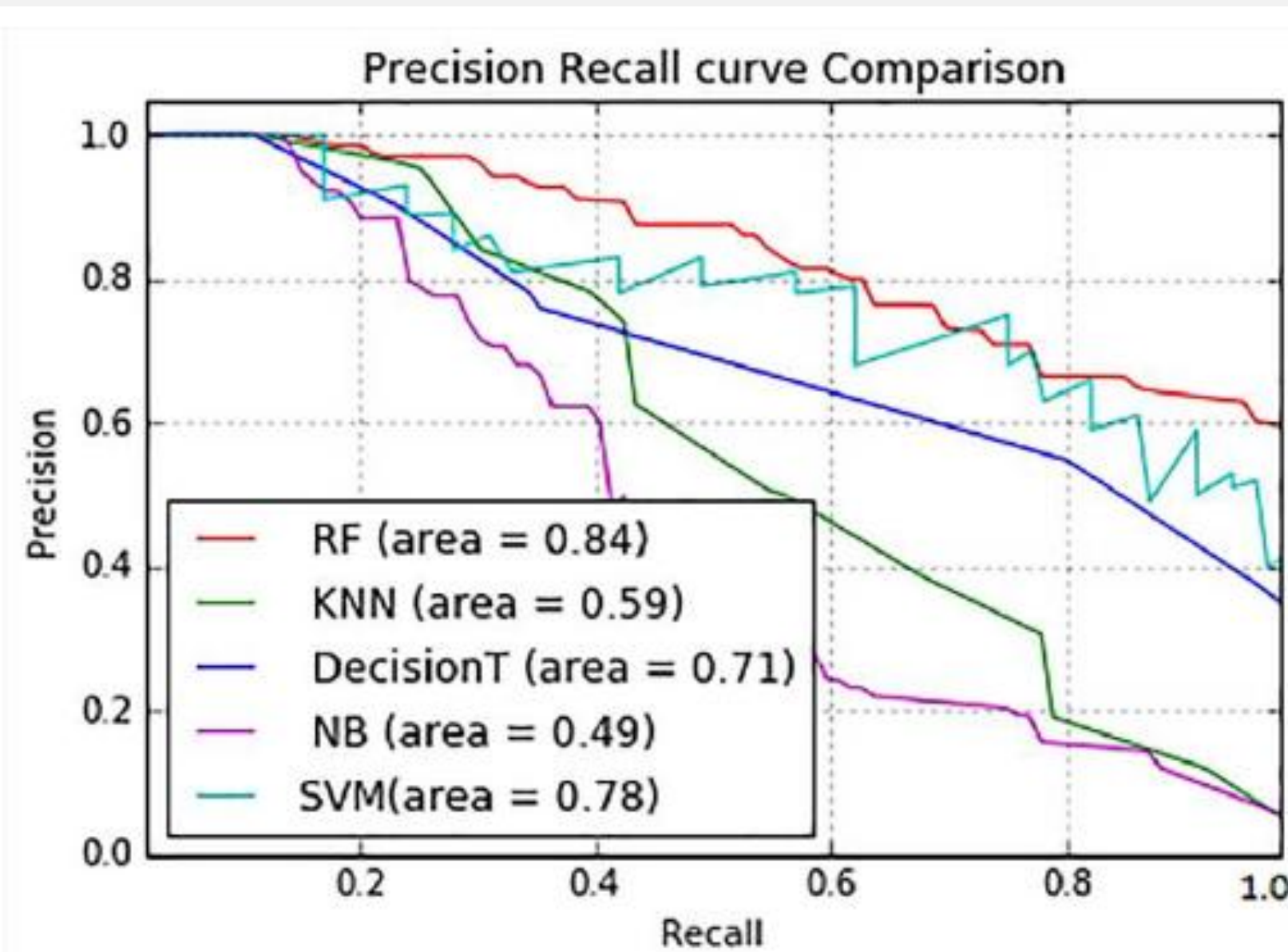




- We want models that achieve better than the baseline. The baseline precision is lower if there is a larger class skew.



- A perfect model would achieve a Precision-Recall AUC of 1



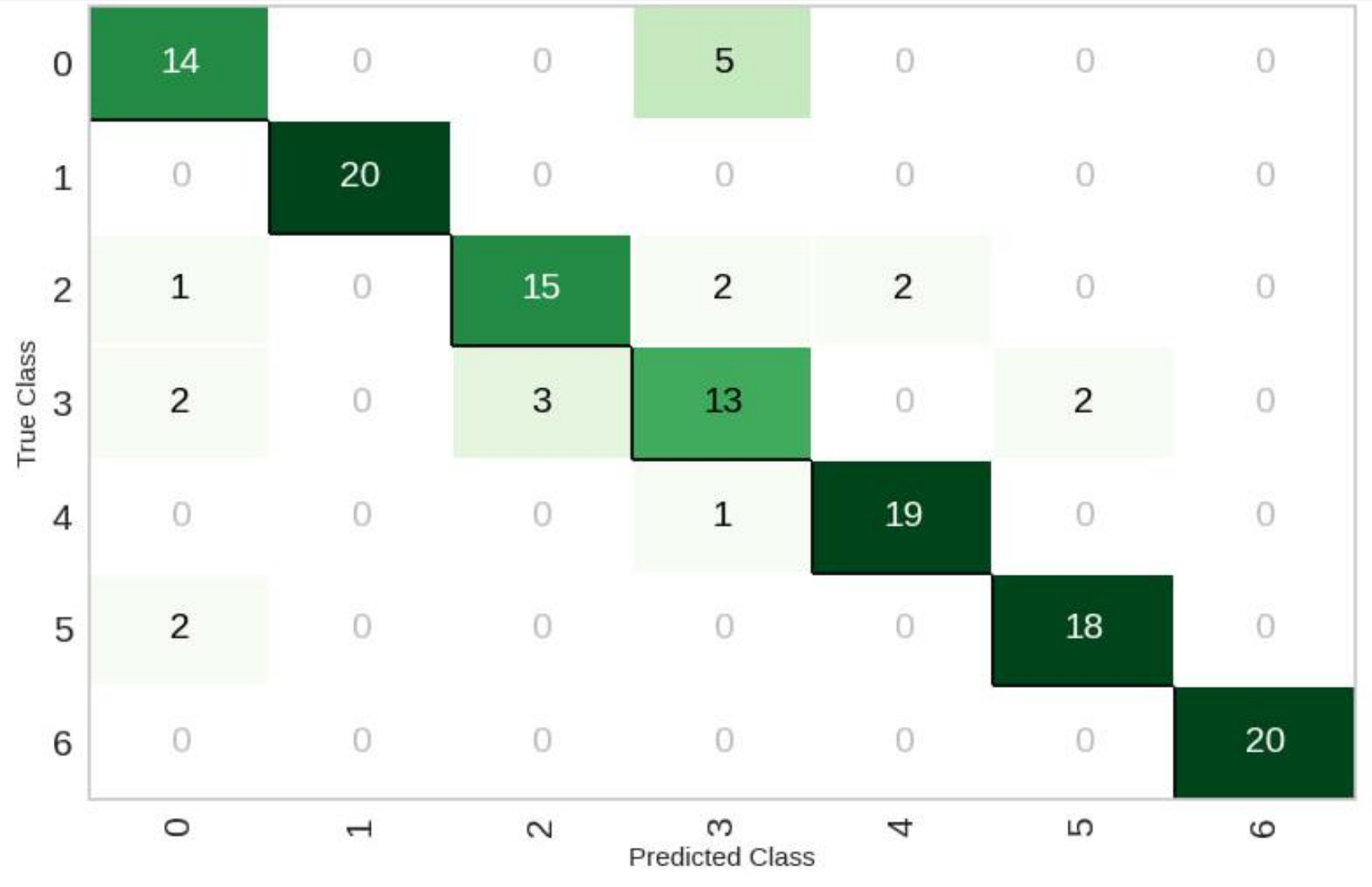
- You can use sklearn's PR plotting function [link](#)
- Using Precision-Recall AUC, we can compare different models.
- We can use scikit-learn's [average precision score](#).
- From the Precision-Recall AUC metrics, we see that the RF model (random forest) has the highest Precision-Recall AUC.

Image from: [link](#)

ROC and Precision-Recall Summary

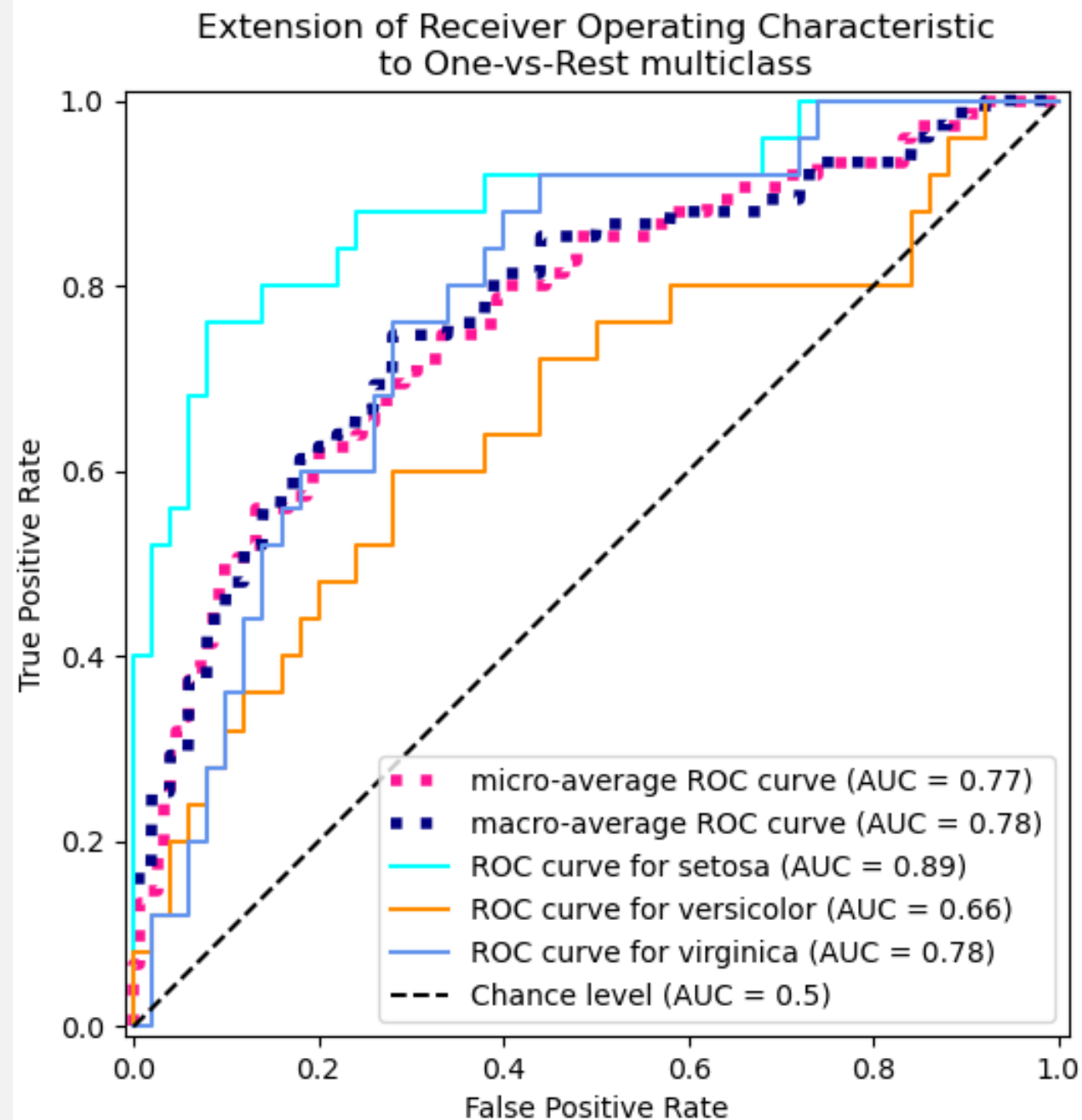
- Both ROC and Precision-recall evaluate the performance of a model at various thresholds.
- ROC measures the *overall* performance of a model at separating the two classes. It is **not** sensitive to the class distribution.
- Precision-Recall is more suitable when you have a large class skew. That is, the positive class is rare.
- Both can be summarized by the area under the curve.
- You can use both ROC-AUC and Precision-Recall AUC to compare among models.

Confusion Matrix Multiclass – Example



See sklearn confusion matrix functions: [link](#) and [link](#)

Image from [link](#)



See guide at [link](#)

Bias-Variance Tradeoff

Simplified Overview

- We will not discuss the math formulation, but give a *simplified* overview.
- Consider an **ML algorithm** that is used for a specific supervised learning problem. Its *generalization* mean-squared-error is decomposed as:

$$MSE = Bias^2 + Variance + Noise$$

- Bias: What is the inherent error of the algorithm with infinite training data? Think of this as the error of the *average* hypothesis $\bar{h}$ that your algorithm learns averaged over many *sampled* datasets (or infinite data). Typically, algorithms with *simple* hypothesis spaces have high bias, and as such can underfit.
- Variance: Consider hypotheses learnt from *different samples* of training data. The variance measures the *spread/deviation* of these hypotheses' predictions from the average hypothesis $\bar{h}$ prediction.
- Noise: This error measures ambiguity due to your data distribution and feature representation. This error is *irreducible*. It is a property of the specific learning task. One example is the noise we formulated in linear regression.

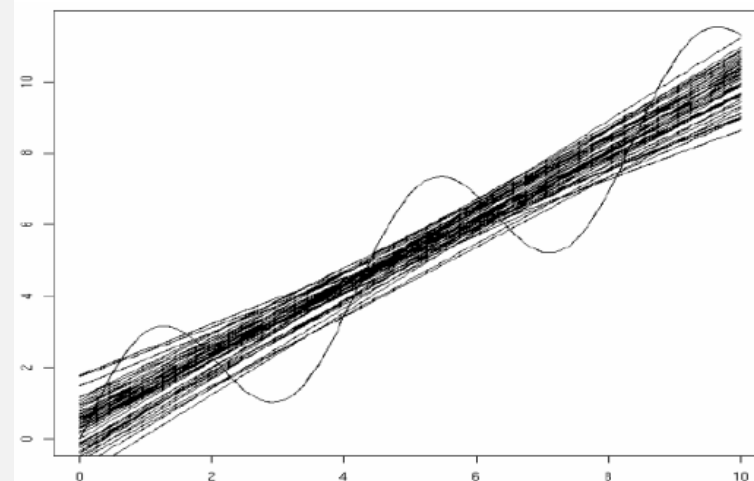
- The error a trained ML **algorithm** makes on a specific learning problem can be decomposed as a sum of: **bias**, **variance**, and **noise**.
- We will not discuss the exact math formulation for this but we will explain the concepts.

$$\underbrace{E_{\mathbf{x},y,D} \left[(h_D(\mathbf{x}) - y)^2 \right]}_{\text{Expected Test Error}} = \underbrace{E_{\mathbf{x},D} \left[(h_D(\mathbf{x}) - \bar{h}(\mathbf{x}))^2 \right]}_{\text{Variance}} + \underbrace{E_{\mathbf{x},y} \left[(\bar{y}(\mathbf{x}) - y)^2 \right]}_{\text{Noise}} + \underbrace{E_{\mathbf{x}} \left[(\bar{h}(\mathbf{x}) - \bar{y}(\mathbf{x}))^2 \right]}_{\text{Bias}^2}$$

- Noise: This error measures ambiguity due to your data distribution and feature representation. This error is irreducible. It is a property of the specific learning task.
- Bias: What is the inherent error that you obtain from your algorithm even with infinite training data? This is due to your model being "biased" to a particular kind of solution (e.g. linear classifier). High bias can cause an algorithm to miss the relevant relations between features and outputs (**underfitting**).
- Variance: Captures how much your algorithm changes if you train on a different training set. How "over-specialized" is your algorithm to a particular training set (**overfitting**)? If we have the best possible model for our training data, how far off are we from the average classifier?

Bias-Variance Dilemma – 1 Feature Practical Illustration

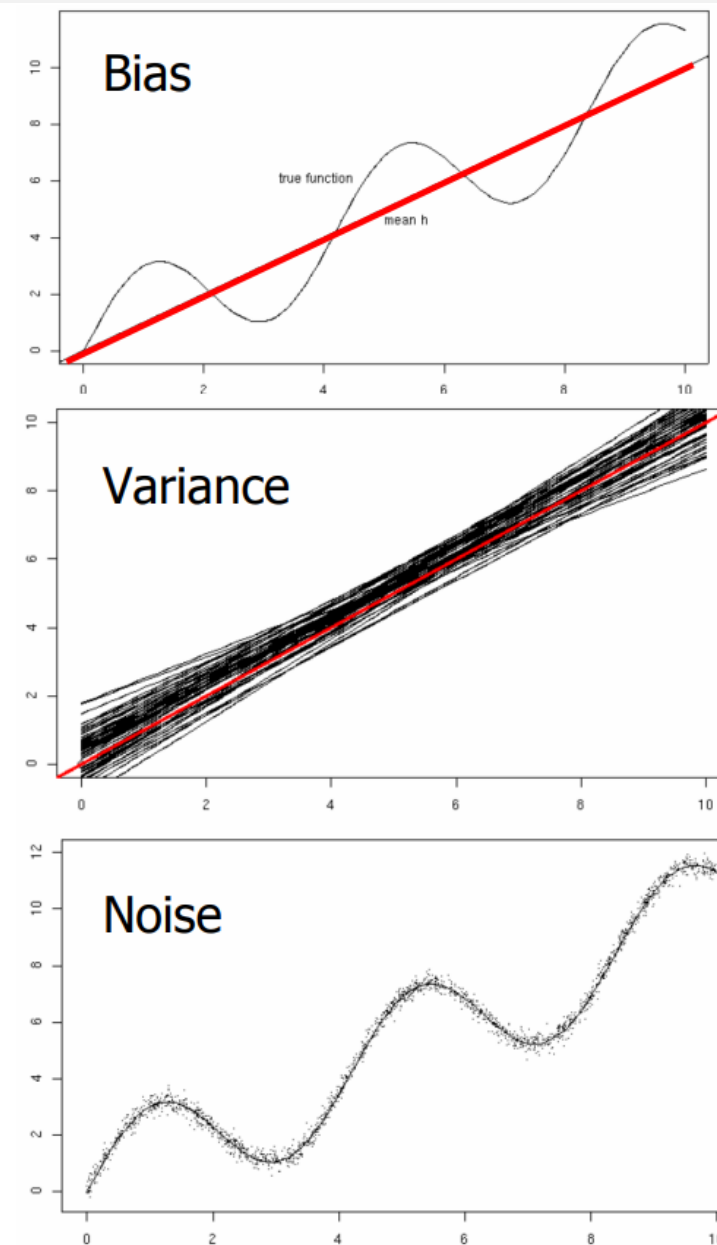
- Consider 1-feature regression as shown. The data distribution is based on the non-linear function *with added noise*.
- Suppose we want to use LinearRegression algo.
- Let's say we sampled 50 different training sets of 20 examples each.
- Error of LinearReg algo:
Bias (the *average* line)
Variance (the deviation from mean line)
Noise (irreducible error)



50 fits (20 examples each)

Image from: [link](#)

=



Bias-Variance Dilemma – Low Bias, High Bias, Low Variance, High Variance

- Below we show a true function (black) being approximated by three polynomial functions of degree 1, 5, and 20.
- We plot various fits for each polynomial, then plot the *average* hypothesis (yellow fit) and its error (bias red dotted line), and variance which is deviation of hypotheses from average hypothesis (blue dotted line).
- Degree 1 has high bias low variance. Degree 5 has low bias low variance. Degree 20 has low bias high variance.

Polynomial models of different degrees fit on random data

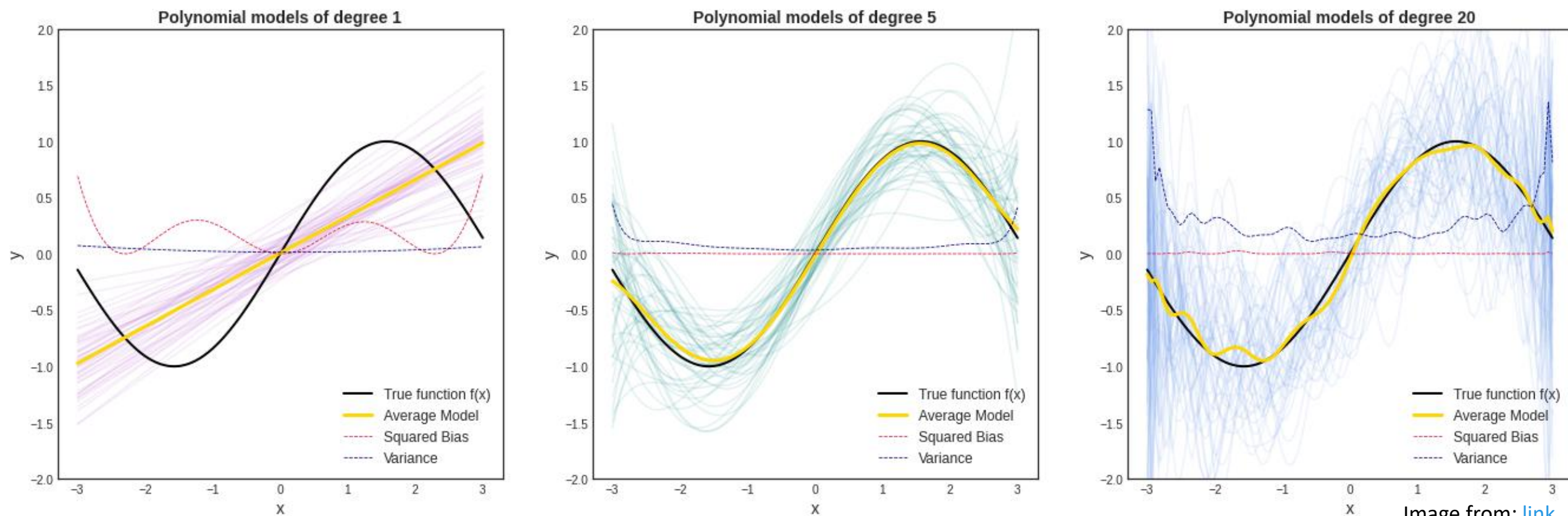
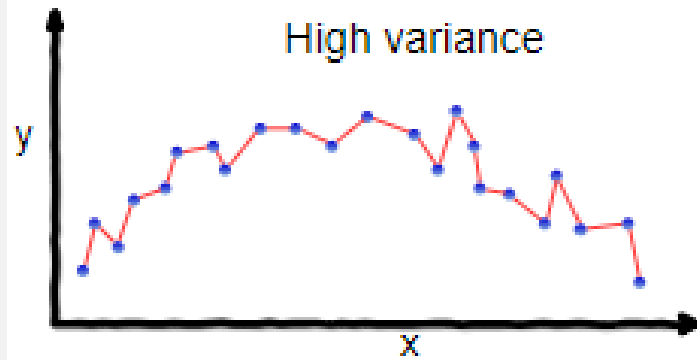


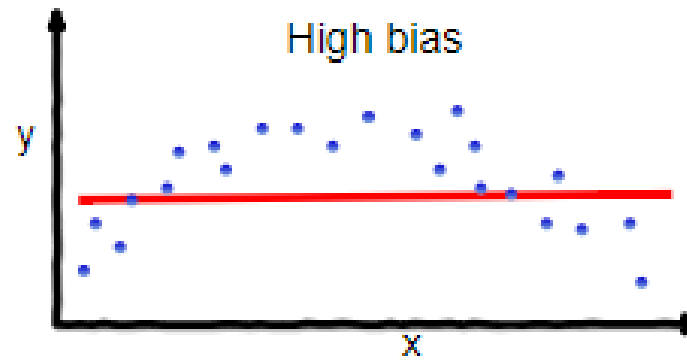
Image from: [link](#)

Bias-Variance Dilemma – Overfitting and Underfitting

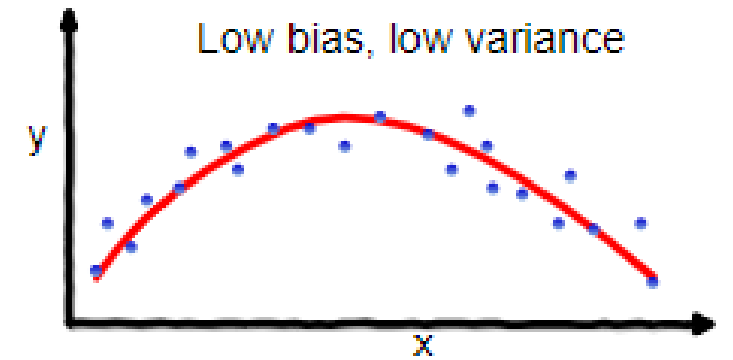
- It's important to note that concepts of high/low bias/variance is dependent on the specifics of the problem:
 - Left algorithm (complex) has low-bias high-variance (maybe overfitting on this specific problem).
 - Middle algorithm (simple) has high-bias low-variance (definitely underfitting on this specific problem).
 - Right algorithm has low-bias low-variance (good generalization on this specific problem).
 - High variance can cause overfitting. High bias can cause underfitting.
- For a given supervised learning problem, we *want* to use an algorithm that has low bias and low variance on that problem.
- For some problems, a *simple* algorithm achieves this. For other problems, we may need increased complex algorithms.



overfitting



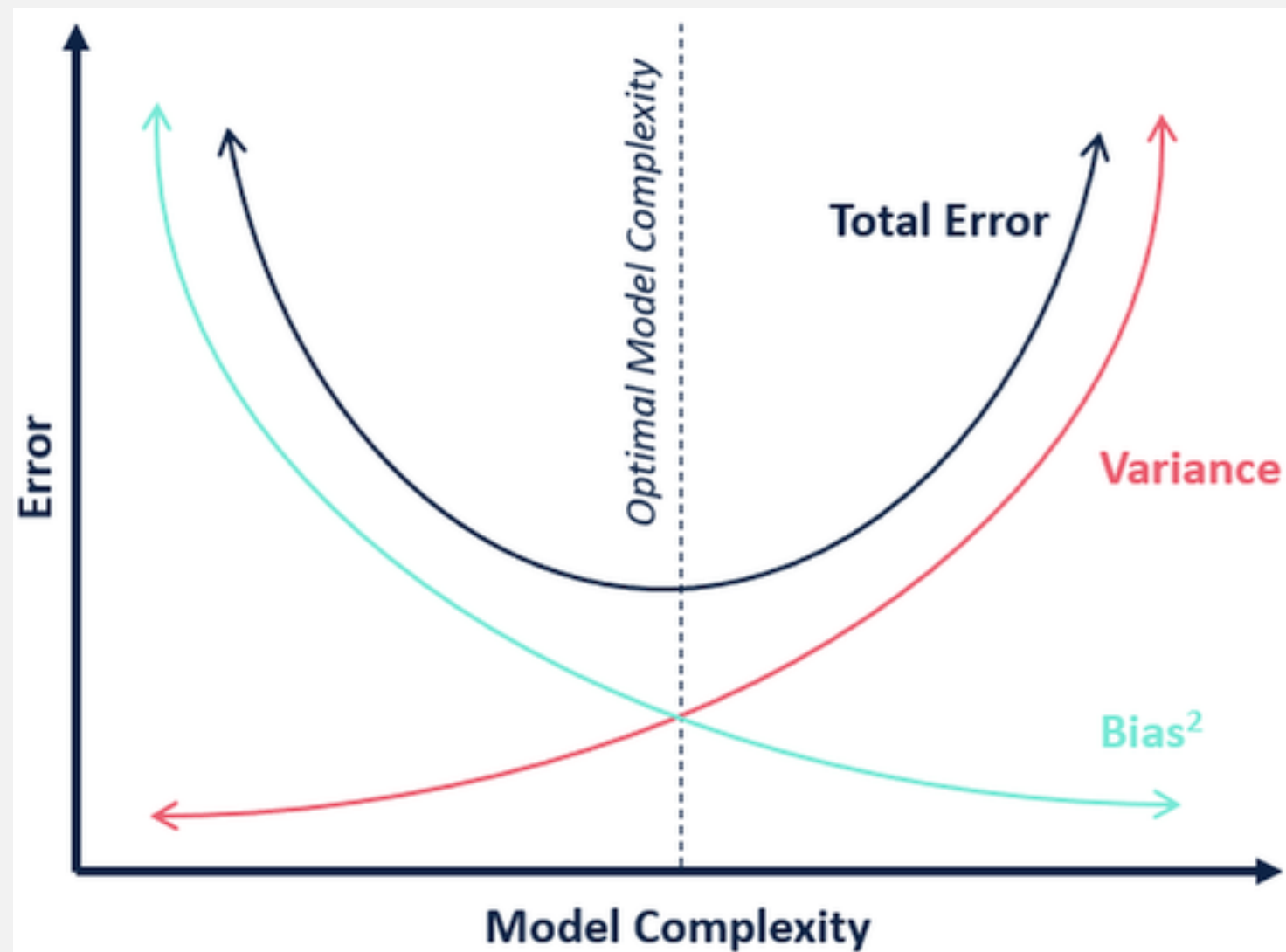
underfitting



Good balance

Bias-Variance Tradeoff

- For a given supervised learning problem, reducing the bias tends to increase the variance, and vice versa.

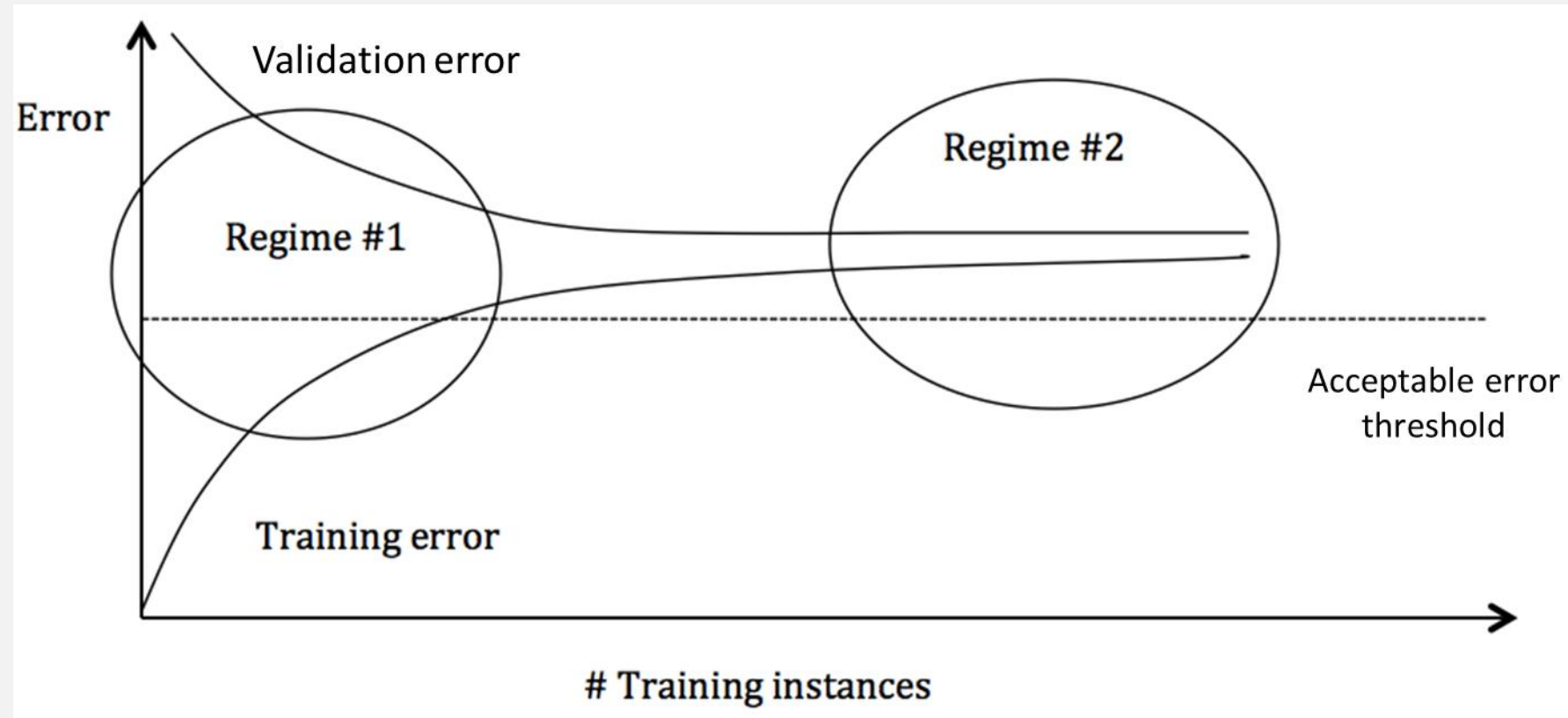


- Bias can be reduced by increasing the complexity of the hypothesis space (e.g. linear to non-linear), but this leads to increased variance.
- Variance can be reduced by decreasing the complexity of the hypothesis space, but this leads to increased bias.
- High variance can cause overfitting. High bias can cause underfitting.
- For the given problem, there is a sweet-spot hypothesis space with low total-error.

Image from: [link](#)

Bias-Variance Scenarios

- Suppose we have an ML algorithm. Suppose we plot the training error of fitted model, and the test error.
- We plot the error curves for increasing size of training data.
- Suppose we get the plotted situation on the right.
- We specify an acceptable error threshold.



- Regime #1: training error is low, but test error is too high. This could be a sign of **overfitting**, so we can try less complex algorithm (reduce variance).
- Regime #2: training error and test error are close, but both are too high. This could be a sign of **underfitting**, so we can try a more complex algorithm (reduce bias).

Practical Example

Model Selection Example [1]

- I grabbed a binary classification [dataset](#) from Kaggle with $m = 768$ examples and $n = 8$ features.
- The task is to predict if a patient has diabetes given their features.

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	127	40	35	168	43.4	0.288	33	1

- I stratify split the dataset into train-test sets with 80-20 ratios. Train set 614 examples, test set 154 examples.
- I plan to do 10-fold CV with the training set.

Model Selection Example [2]

- This is my pool of ML algorithms (some simple and some complex algorithms):

```
1  from sklearn.linear_model import LogisticRegression
2  from sklearn.tree import DecisionTreeClassifier
3  from sklearn.neighbors import KNeighborsClassifier
4  from sklearn.ensemble import RandomForestClassifier
5
6  ml_pool = {
7      'lr1': LogisticRegression(penalty='none'),
8      'lr2': LogisticRegression(penalty='l2', C=0.75),
9      'lr3': LogisticRegression(penalty='l2', C=0.5),
10
11      'dt1': DecisionTreeClassifier(max_features='sqrt', min_samples_leaf=2, random_state=0),
12      'dt2': DecisionTreeClassifier(max_features='log2', min_samples_leaf=2, random_state=0),
13
14      '3nn': KNeighborsClassifier(n_neighbors=3),
15      '5nn': KNeighborsClassifier(n_neighbors=5),
16
17      'rf1': RandomForestClassifier(n_estimators=100, random_state=0),
18      'rf2': RandomForestClassifier(n_estimators=1000, random_state=0),
19
20      'NN2H': instantiate_nn2hmodel,
21      'NN3H': instantiate_nn3hmodel
22  }
```

Note these NN models are functions

Model Selection Example [2]

- The NN models are defined inside custom functions. This is because when we do 10-fold CV, we need to train 10 NN models from scratch. So, we need to reset the model weights and this is one way to do that: just define a new model. If we don't do this, then the network would train on ALL folds.
- We don't need to do this for scikit-learn methods, because calling fit in scikit-learn resets the model and then trains.

```
1  import tensorflow as tf
2  from tensorflow import keras
3  from tensorflow.keras import layers
4
5  def instantiate_nn3hmodel()
6      nn3hmodel = keras.Sequential(
7          [
8              layers.BatchNormalization(input_shape=(8,)),
9              layers.Dense(128, activation="relu", name="layer1"),
10             layers.BatchNormalization(),
11             layers.Dense(128, activation="relu", name="layer2"),
12             layers.BatchNormalization(),
13             layers.Dense(128, activation="relu", name="layer3"),
14             layers.BatchNormalization(),
15             layers.Dense(1, activation='sigmoid', name="output"),
16         ]
17     )
18
19     opt = keras.optimizers.Adam(lr=0.001, amsgrad=True)
20     nn3hmodel.compile(optimizer=opt, loss='binary_crossentropy')
21     nn3hmodel.build()
22     nn3hmodel.summary()
23
24     return nn3hmodel
```

- Apply 10-fold CV on the models in the pool and gather the 10-fold results.

```

1  from sklearn.model_selection import StratifiedKFold
2  from sklearn.preprocessing import StandardScaler
3  from sklearn import metrics
4
5  scoring_funcs = [metrics.roc_auc_score, metrics.average_precision_score]
6
7  skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=0)
8
9  eval_results = []
10 fold_num = 0
11 for train_folds, test_folds in skf.split(X_train, y_train):
12     X_train_fold, X_test_fold = X_train[train_folds], X_train[test_folds]
13     y_train_fold, y_test_fold = y_train[train_folds], y_train[test_folds]
14
15     # pre-process
16     scaler = StandardScaler()
17     scaler.fit(X_train_fold)
18
19     X_train_fold = scaler.transform(X_train_fold)
20     X_test_fold = scaler.transform(X_test_fold)
21
22     # train and evaluate models in pool
23     for model in ml_pool:
24         if model in ['NN2H', 'NN3H']:
25             nn_model = ml_pool[model]()
26             nn_model.fit(X_train_fold, y_train_fold, batch_size=128, verbose=0, epochs=2000)
27             y_pred_on_test_fold = nn_model.predict(X_test_fold)[: ,0]
28         else:
29             ml_pool[model].fit(X_train_fold, y_train_fold)
30             y_pred_on_test_fold = ml_pool[model].predict_proba(X_test_fold)[: ,1]
31
32             model_eval_results = []
33             for scoring_metric in scoring_funcs:
34                 model_eval_results.append(scoring_metric(y_test_fold, y_pred_on_test_fold))
35
36             eval_results.append([model, fold_num] + model_eval_results) # name of model and its fold evaluation results
37
38 fold_num += 1

```

I used ROC-AUC and Precision-Recall AUC.

Use K-fold stratified splitting

Standard scale using the training fold
(topic we do not discuss in this course)

The nn models are instantiated to reset the weights

Train on train folds and evaluate on test fold

- Analyze the 10-fold performance results. I can create a pandas dataframe for the results.

eval_df				
	Model	Fold Index	ROC-AUC	PR-AUC
5	3nn	0	0.817045	0.667290
16	3nn	1	0.624432	0.420479
27	3nn	2	0.700000	0.520856
38	3nn	3	0.763636	0.573857
49	3nn	4	0.755952	0.559647
60	3nn	5	0.724405	0.532592
71	3nn	6	0.854762	0.753365
82	3nn	7	0.833333	0.666912
93	3nn	8	0.687500	0.467528
104	3nn	9	0.761310	0.558693
6	5nn	0	0.785795	0.599731
17	5nn	1	0.692045	0.487492
28	5nn	2	0.754545	0.585952
39	5nn	3	0.831250	0.669664

- We can also summarize the results by using pandas grouping functions.

```
eval_df = pd.DataFrame(data=eval_results,
                       columns=['Model', 'Fold Index', 'ROC-AUC', 'PR-AUC'])
eval_df = eval_df.sort_values(['Model', 'Fold Index'])
```

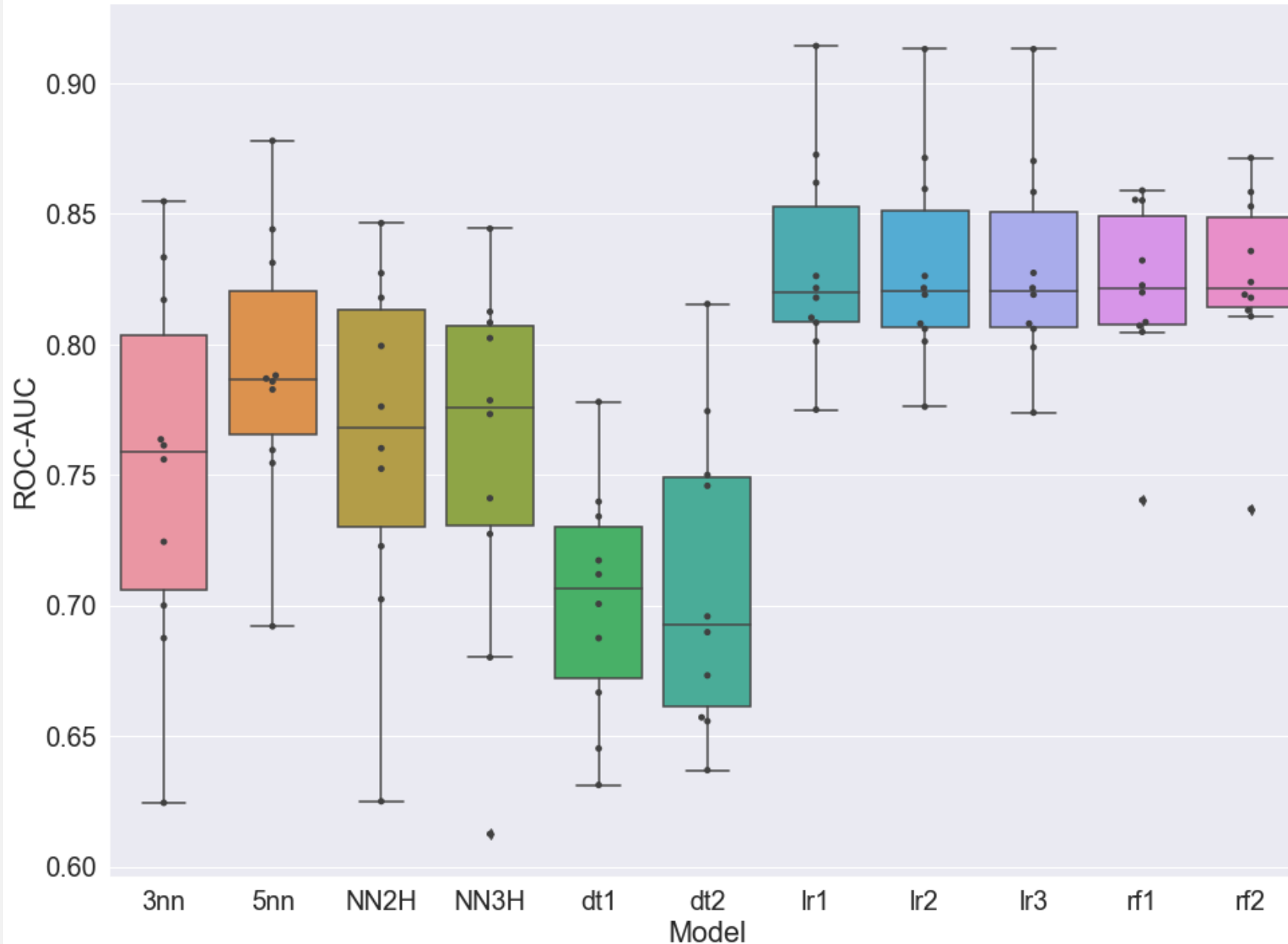
```
eval_df.groupby(['Model']).describe()[['ROC-AUC', 'PR-AUC']].round(2)
```

	ROC-AUC								PR-AUC							
	count	mean	std	min	25%	50%	75%	max	count	mean	std	min	25%	50%	75%	max
Model																
3nn	10.0	0.75	0.07	0.62	0.71	0.76	0.80	0.85	10.0	0.57	0.10	0.42	0.52	0.56	0.64	0.75
5nn	10.0	0.79	0.05	0.69	0.77	0.79	0.82	0.88	10.0	0.62	0.07	0.49	0.59	0.61	0.68	0.73
NN2H	10.0	0.76	0.07	0.62	0.73	0.77	0.81	0.85	10.0	0.66	0.09	0.52	0.60	0.65	0.72	0.78
NN3H	10.0	0.76	0.07	0.61	0.73	0.78	0.81	0.84	10.0	0.63	0.10	0.44	0.56	0.65	0.69	0.78
dt1	10.0	0.70	0.05	0.63	0.67	0.71	0.73	0.78	10.0	0.51	0.05	0.42	0.47	0.51	0.55	0.59
dt2	10.0	0.71	0.06	0.64	0.66	0.69	0.75	0.82	10.0	0.53	0.08	0.45	0.47	0.50	0.56	0.72
lr1	10.0	0.83	0.04	0.78	0.81	0.82	0.85	0.91	10.0	0.74	0.08	0.63	0.68	0.75	0.78	0.87
lr2	10.0	0.83	0.04	0.78	0.81	0.82	0.85	0.91	10.0	0.74	0.08	0.64	0.68	0.75	0.78	0.87
lr3	10.0	0.83	0.04	0.77	0.81	0.82	0.85	0.91	10.0	0.74	0.08	0.63	0.68	0.75	0.77	0.87
rf1	10.0	0.82	0.04	0.74	0.81	0.82	0.85	0.86	10.0	0.71	0.07	0.60	0.66	0.72	0.77	0.81
rf2	10.0	0.82	0.04	0.74	0.81	0.82	0.85	0.87	10.0	0.72	0.09	0.56	0.67	0.73	0.77	0.84

- We can also visually summarize the results in a plot using the [seaborn](#) package. seaborn is a python plotting package that is built on top of matplotlib.

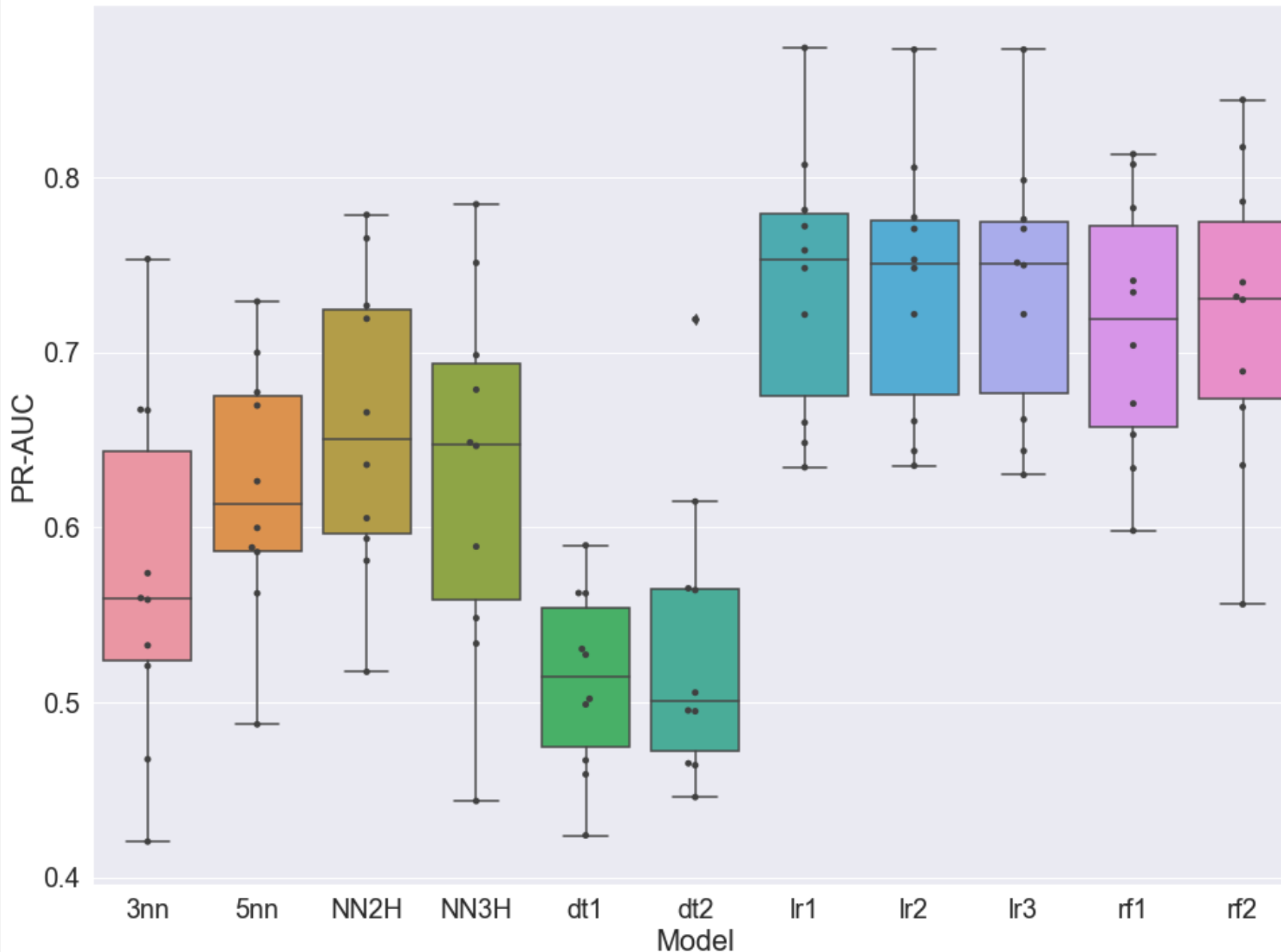
```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 sns.set_context("paper")
4 sns.set(font_scale=1.8)
5
6 plt.figure(figsize=(16, 12))
7
8 ax = sns.boxplot(x="Model", y="ROC-AUC", data=eval_df)
9 ax = sns.swarmplot(x="Model", y="ROC-AUC", data=eval_df, color=".25")
10
11 plt.title('10-fold CV results on ROC-AUC')
12
13 plt.figure(figsize=(16, 12))
14 ax = sns.boxplot(x="Model", y="PR-AUC", data=eval_df)
15 ax = sns.swarmplot(x="Model", y="PR-AUC", data=eval_df, color=".25")
16
17 plt.title('10-fold CV results on PR-AUC')
```

10-fold CV results on ROC-AUC



- LogisticRegression seems to do well on this task based on ROC-AUC.

10-fold CV results on PR-AUC



- LogisticRegression also seems to do well on this task based on PR-AUC.

- So, based on 10-fold results, I'd select 'lr1'. Now I re-train this on the entire training set. Then evaluate on the test set.

```
1  best_model = LogisticRegression(penalty='l2')
2
3  # pre-process
4  scaler = StandardScaler()
5  scaler.fit(X_train)
6
7  X_train_scaled = scaler.transform(X_train)
8  X_test_scaled = scaler.transform(X_test)
9
10 # retrain on entire training
11 best_model.fit(X_train_scaled, y_train)
12
13 #evaluate on test set
14 y_pred_on_test = best_model.predict_proba(X_test_scaled)[:,-1]
15 test_roc = metrics.roc_auc_score(y_test, y_pred_on_test)
16 test_pr = metrics.average_precision_score(y_test, y_pred_on_test)
17
18 print('Test ROC-AUC: {:.2f}. Test PR-AUC: {:.2f}'.format(test_roc, test_pr))
19
```

- This gives a result of “Test ROC-AUC: 0.823. Test PR-AUC: 0.692”.